

Vibra - Trabajo de Fin de Ciclo



Nicolás Buquet

Jorge López



Índice

1. Introducción	1
2. Justificación y objetivos del proyecto	2
2.1. Justificación	2
2.1.1. Justificación de las Tecnologías utilizadas	2
2.2. Objetivos	3
2.2.1. Objetivo general	3
2.2.2. Objetivos específicos	3
3. Metodología del trabajo	4
3.1. Planificación de tareas del proyecto y asignación de tareas	4
3.2. Dependencias entre tareas	5
3.3. Herramientas de comunicación y reuniones	7
4. Desarrollo del proyecto	8
4.1. Descripción de tecnologías y herramientas	8
4.1.1. Backend	8
4.1.2. Servidor y despliegue	11
4.1.3. Frontend	12
4.2. Diseño y esquemas de las tecnologías del proyecto	13
4.2.1. Esquema de despliegue	13
4.2.2. Diagrama de clases del modelo de datos	14
4.2.3. Esquema de la base de datos	15
4.2.4. Diagrama de comunicación entre contenedores	16
4.2.5. Diagrama de componentes	17

4.2.6.	Diagrama de rutas	18
4.2.7.	Diagrama de contextos	19
4.3.	Implementación, gestión y configuración del proyecto	20
4.3.1.	Base de datos	20
4.3.2.	Backend	22
4.3.3.	Servidor y despliegue	39
4.3.4.	Frontend	46
4.3.5.	Pruebas funcionales del sistema	53
4.3.6.	Errores encontrados y resolución	58
5.	Conclusiones	62
6.	Bibliografía	63



1. Introducción

El presente proyecto consiste en el desarrollo de una red social centrada exclusivamente en la música y el bienestar digital. En un contexto donde el uso excesivo y poco saludable de las tecnologías digitales se ha convertido en una preocupación creciente, esta aplicación busca ofrecer un espacio que combine interacción social con herramientas que fomenten el equilibrio y la salud mental, utilizando la música como vehículo principal.

El enfoque del proyecto es doble: por un lado, crear una experiencia de usuario agradable, intuitiva y minimalista que refuerce un uso saludable de las redes sociales; por otro, utilizar tecnologías modernas y eficientes para su desarrollo e implementación. La estética de la aplicación ha sido cuidadosamente diseñada para transmitir calma y claridad, coherente con la imagen de marca creada para la plataforma.

Desde el punto de vista técnico, el proyecto está construido con tecnologías avanzadas para asegurar su rendimiento, flexibilidad y escalabilidad. Se emplea Quarkus para un backend completamente reactivo, PostgreSQL como sistema gestor de bases de datos y React, combinado con Material UI y Sass, para un frontend moderno y adaptable. Finalmente, el despliegue se realiza sobre un clúster Kubernetes configurado con Kind en un entorno ligero basado en Rocky Linux minimal, simulando condiciones reales de producción.

Este documento recoge el proceso completo de desarrollo, desde la planificación inicial hasta la implementación final, incluyendo las pruebas necesarias para validar su correcto funcionamiento.



2. Justificación y objetivos del proyecto

2.1. Justificación

La elección de desarrollar una red social centrada en la música y el bienestar digital responde a la creciente necesidad de ofrecer espacios digitales que promuevan hábitos saludables y un uso consciente de las tecnologías digitales. Actualmente, numerosas plataformas digitales tienden a incentivar el consumo excesivo de contenido y generan problemas relacionados con la ansiedad, el estrés y la desconexión emocional de los usuarios.

En este contexto, surge la idea de crear una aplicación minimalista y funcional, diseñada específicamente para fomentar el bienestar digital mediante la música, un recurso ampliamente reconocido por sus beneficios para la salud mental y emocional.

2.1.1. Justificación de las Tecnologías utilizadas

- **Quarkus:** Permite crear un backend reactivo con alto rendimiento, escalable y nativo de kubernetes. Adecuado para aplicaciones web modernas.
- **PostgreSQL:** Base de datos robusta y fiable, eficiente en el manejo de relaciones complejas y compatible con programación reactiva.
- **React, Material UI y Sass:** Facilitan el desarrollo de una interfaz visualmente atractiva, modular, escalable y de fácil mantenimiento.
- **TypeScript:** Aporta tipado estático al desarrollo frontend, mejorando la mantenibilidad y reduciendo errores en tiempo de ejecución. Su integración con React permite un desarrollo más estructurado y seguro.
- **Kubernetes (sobre Rocky Linux):** Simula fielmente un entorno real de producción, facilitando despliegues automáticos, escalabilidad y monitorización.



2.2. Objetivos

2.2.1. Objetivo general

Desarrollar una aplicación web tipo red social que promueva el bienestar digital y la interacción social a través de la música, garantizando un rendimiento eficiente y una experiencia de usuario óptima.

2.2.2. Objetivos específicos

- Implementar un backend reactivo utilizando el framework Quarkus con Reactive Panache, optimizando la capacidad de respuesta y el rendimiento del servidor.
- Desplegar la aplicación en un entorno de producción simulado mediante Kubernetes, sobre una máquina virtual Rocky Linux minimal.
- Crear una interfaz minimalista, intuitiva y visualmente agradable para mejorar la experiencia del usuario.
- Utilizar React, Material UI y Sass para construir un frontend modular, escalable y atractivo visualmente.
- Incorporar TypeScript en el desarrollo frontend para mejorar la mantenibilidad del código, reducir errores en tiempo de ejecución y facilitar la integración con React mediante tipado estático.
- Realizar pruebas completas para asegurar el correcto funcionamiento y estabilidad del sistema.



3. Metodología del trabajo

3.1. Planificación de tareas del proyecto y asignación de tareas

La planificación del proyecto se ha estructurado en varias fases, cada una compuesta por tareas específicas que permiten avanzar de manera ordenada y eficiente hacia los objetivos finales. La estimación de tiempos se ha realizado en semanas, cubriendo el periodo comprendido entre marzo y junio. La asignación de tareas se ha realizado en función de la especialización y experiencia de cada miembro del equipo, asegurando así una mayor eficiencia y calidad en el desarrollo del proyecto. La siguiente tabla resume las tareas principales, el responsable de cada una y la duración estimada:

Área	Responsable	Tiempo estimado
Planificación	Jorge y Nico	2 semanas
Diseño	Jorge y Nico	1 semana
Frontend	Jorge	11 semanas
Backend	Nico	9 semanas
Base de datos	Nico	1 semana
Despliegue	Nico	1 semana
Pruebas	Jorge y Nico	1 semana

Cuadro 1: Planificación

3.2. Dependencias entre tareas

Las principales dependencias identificadas son:

- **Planificación y diseño:** Las tareas de definición de requisitos y diseño inicial son previas y necesarias para el desarrollo tanto del frontend como del backend.
- **Backend y base de datos:** El desarrollo del backend y el modelado de la base de datos deben estar avanzados para permitir la integración y pruebas del frontend.
- **Frontend:** El desarrollo de las distintas vistas y componentes del frontend depende parcialmente de la disponibilidad de la API backend, aunque algunas tareas pueden realizarse en paralelo.
- **Despliegue:** La configuración de la infraestructura y el despliegue en Kubernetes requieren que los servicios de backend y frontend estén implementados y validados.
- **Pruebas:** Las pruebas funcionales y de integración se realizan una vez desplegados todos los servicios y completadas las tareas de desarrollo.
- **Documentación:** La elaboración de la memoria y documentación se realiza de forma continua, pero se intensifica en la fase final tras la validación del sistema.

Estas dependencias se reflejan en el siguiente diagrama de Gantt (siguiente página), que permite visualizar de forma clara la secuencia, solapamiento y duración estimada de cada tarea, así como las relaciones entre ellas:

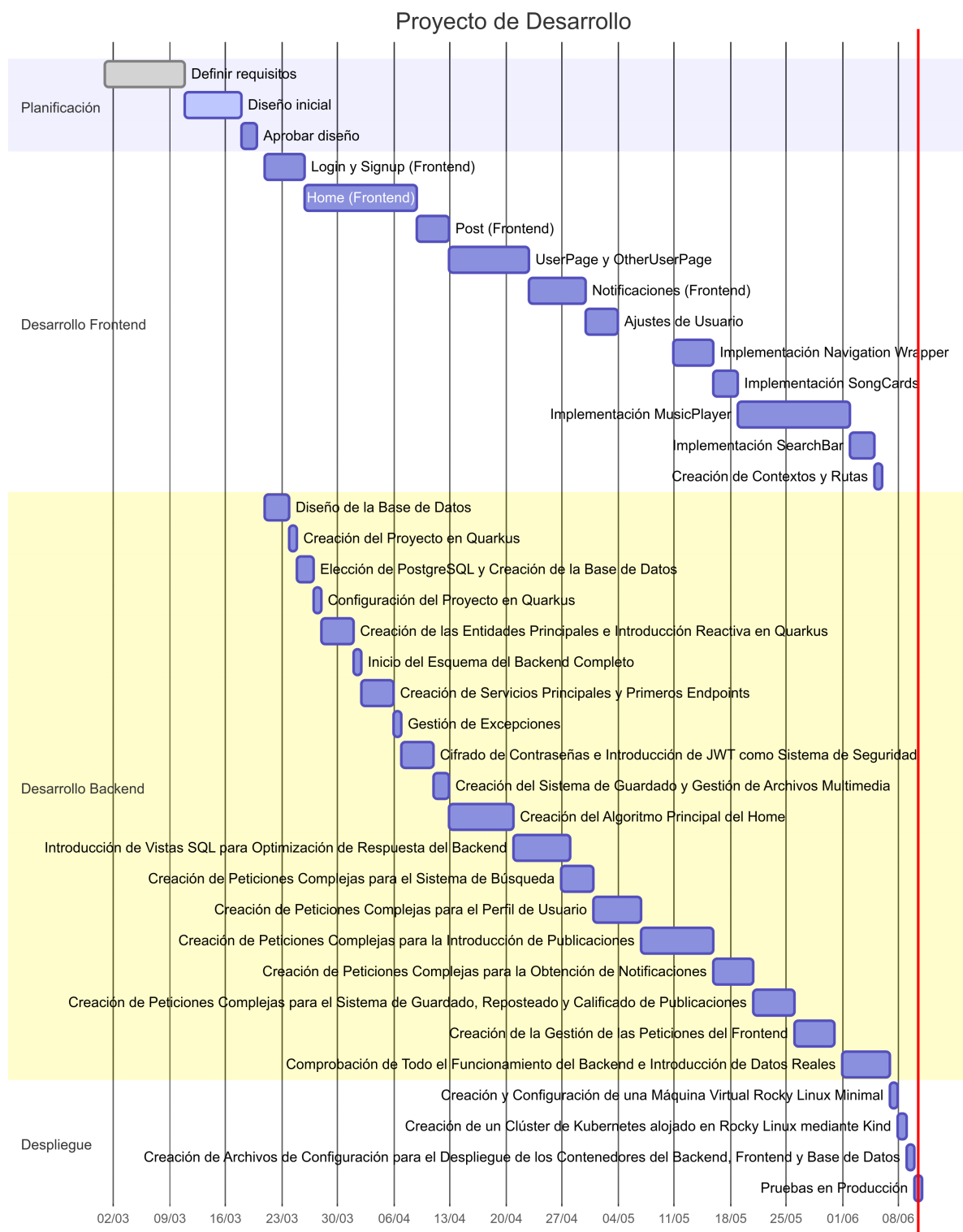


Figura 1: Diagrama de Gantt



3.3. Herramientas de comunicación y reuniones

Para la comunicación y coordinación del equipo durante el desarrollo del proyecto, se ha utilizado principalmente la plataforma Discord, que ha permitido realizar reuniones virtuales semanales, compartir avances, resolver dudas y coordinar tareas de manera eficiente y en tiempo real. Además, cada dos semanas se han realizado reuniones presenciales en las tutorías con el profesor responsable, lo que ha facilitado la revisión del progreso, la resolución de problemas más complejos y la toma de decisiones clave.

Esta combinación de reuniones virtuales y presenciales ha permitido mantener una comunicación fluida, un seguimiento constante del proyecto y una rápida respuesta ante cualquier incidencia o necesidad de ajuste en la planificación.

4. Desarrollo del proyecto

4.1. Descripción de tecnologías y herramientas

4.1.1. Backend

Para el desarrollo del backend se ha optado por el framework Quarkus, una tecnología moderna diseñada específicamente para entornos nativos en la nube. Su arquitectura ligera, junto con un tiempo de arranque muy reducido y un bajo consumo de memoria, lo convierten en una opción ideal para aplicaciones desplegadas en contenedores y orquestadas con Kubernetes.

Además de estas ventajas de rendimiento, Quarkus destaca por su compatibilidad con compilación nativa a través de GraalVM, lo que permite generar ejecutables autónomos extremadamente eficientes. Esta capacidad es especialmente útil en entornos de alta concurrencia, como el de este proyecto, donde se requiere que la infraestructura sea ágil, escalable y fácilmente replicable.

Traditional Frameworks

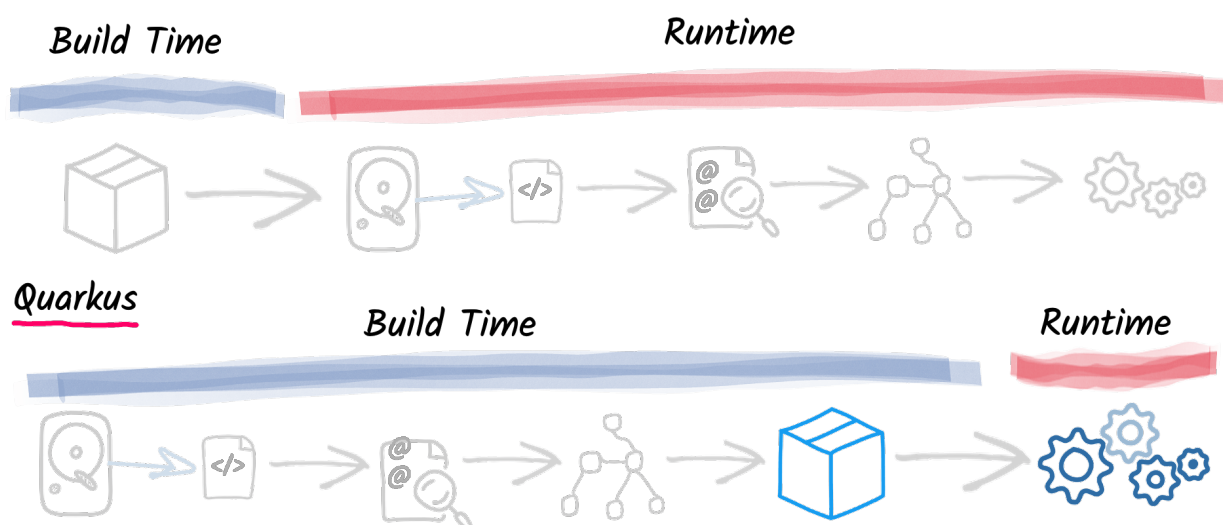


Figura 2: Funcionamiento de Quarkus

Más allá del entorno de ejecución, una de las razones clave para elegir Quarkus fue su soporte nativo para la programación reactiva, un enfoque que permite gestionar múltiples operaciones simultáneas de forma eficiente. Esta característica ha sido fundamental para garantizar que el backend responda de manera fluida incluso bajo carga, aprovechando el modelo asíncrono y no bloqueante que caracteriza a este paradigma.

Programación reactiva

La programación reactiva es un paradigma que permite construir sistemas capaces de manejar operaciones de entrada/salida y eventos de forma no bloqueante, mediante un modelo basado en la gestión asíncrona de flujos de datos. Este enfoque es fundamental para aplicaciones que requieren alta concurrencia y baja latencia, como la red social desarrollada en este proyecto.

En un modelo tradicional bloqueante, operaciones lentas como consultas a bases de datos o llamadas a servicios externos pueden detener la ejecución de un hilo hasta completarse, limitando la escalabilidad y eficiencia del sistema. La programación reactiva evita estos bloqueos mediante un **event loop**, que gestiona las operaciones de forma asíncrona, permitiendo que el sistema continúe procesando otras tareas mientras espera las respuestas.

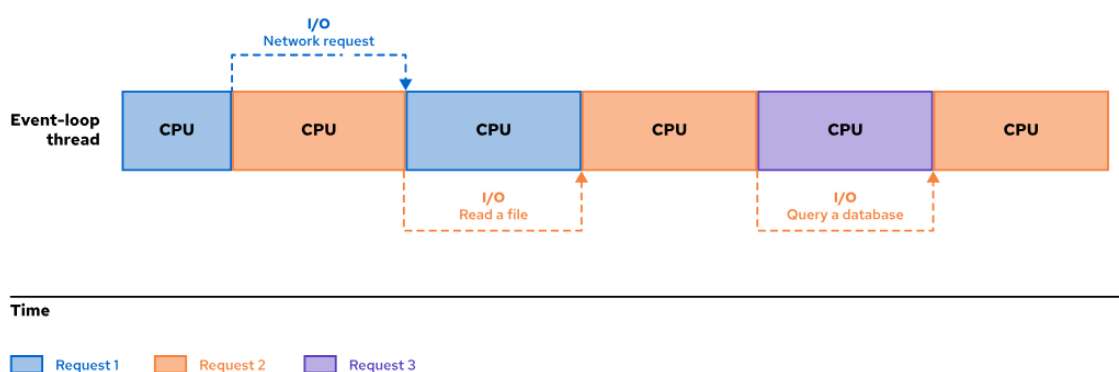


Figura 3: Event-loop thread soportando multiples peticiones

Este mecanismo optimiza el uso de recursos, especialmente en entornos donde el número de conexiones concurrentes es elevado, y es particularmente compatible con la arquitectura basada en microservicios y contenedores implementada en este proyecto.

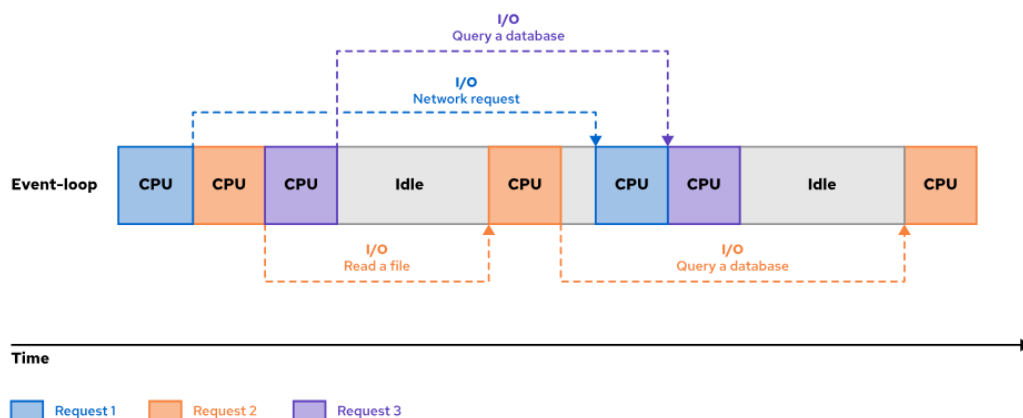


Figura 4: Event-loop soportando multiples peticiones

Autenticación con JWT

La autenticación del backend se ha implementado mediante tokens JWT (JSON Web Tokens), utilizando la extensión `quarkus-smallrye-jwt`. Este sistema permite autenticar y autorizar a los usuarios de forma segura y sin mantener estado en el servidor, lo que se alinea perfectamente con el enfoque reactivo y escalable del proyecto.

El flujo de autenticación es el siguiente: cuando un usuario inicia sesión correctamente, el backend genera un token firmado que contiene información codificada sobre la identidad del usuario (como su nombre o rol). Este token es devuelto al frontend, que lo almacena localmente (por ejemplo, en memoria o en el almacenamiento local del navegador).

En las siguientes peticiones, el frontend incluye el token en la cabecera `Authorization` de tipo `Bearer`. El backend, al recibir la solicitud, valida la firma del token y extrae la identidad del usuario sin necesidad de realizar nuevas consultas a la base de datos, lo que optimiza el rendimiento.

Este sistema, al no requerir sesiones ni almacenamiento persistente del estado de autenticación, se adapta perfectamente a arquitecturas distribuidas, como la que se plantea con el uso de contenedores y Kubernetes.

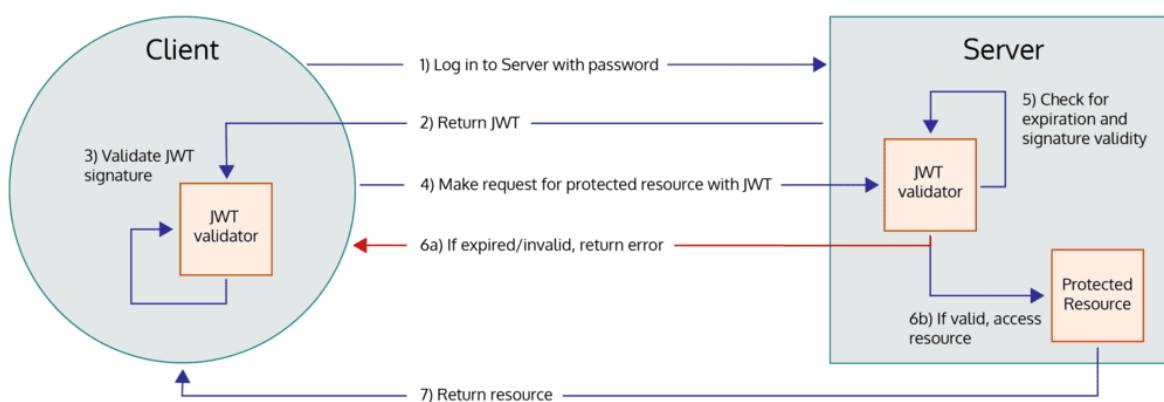


Figura 5: Flujo de autenticación con JWT

Base de datos: PostgreSQL

La base de datos utilizada para la persistencia de la información es **PostgreSQL**, un sistema de gestión relacional potente, fiable y ampliamente utilizado en entornos de producción. PostgreSQL ofrece una gran compatibilidad con estructuras complejas, así como soporte completo para integrarse con sistemas reactivos, lo que lo convierte en una opción adecuada para este proyecto.

4.1.2. Servidor y despliegue

Para el entorno de servidor, se ha elegido **Rocky Linux minimal**, una distribución estable y ligera basada en RHEL. Esta elección proporciona un sistema limpio y optimizado, ideal para configurar un entorno de despliegue controlado y reproducible, sin servicios innecesarios que puedan interferir con el rendimiento o la seguridad.

Sobre esta base se ha utilizado **Kubernetes**, una plataforma de orquestación de contenedores que permite gestionar de manera eficiente aplicaciones distribuidas. Kubernetes facilita tareas como el escalado automático, el balanceo de carga, el reinicio de servicios caídos y la actualización sin interrupciones, cualidades esenciales para aplicaciones modernas.

Para el desarrollo y pruebas, se ha usado la herramienta **Kind (Kubernetes IN Docker)**, que permite crear clústeres de Kubernetes locales dentro de contenedores. Esto ha hecho posible probar todo el entorno en una máquina local antes de migrar a un entorno más complejo. Kind es especialmente útil para validar configuraciones y estructuras de despliegue sin necesidad de recursos cloud.

La construcción y ejecución de contenedores se ha realizado con **Podman**, una alternativa moderna a Docker, sin demonio y compatible con los estándares OCI. Su integración con systemd y su mayor foco en la seguridad lo hacen especialmente adecuado para entornos de servidores Linux como el utilizado en este proyecto.

El despliegue de la aplicación se ha gestionado mediante archivos YAML, donde se han definido los recursos de Kubernetes como Deployments, Services y volúmenes persistentes. Esta separación de responsabilidades en contenedores —frontend, backend y base de datos— permite una arquitectura limpia, modular y escalable, que facilita el mantenimiento a largo plazo.

4.1.3. Frontend

Para el desarrollo del frontend se ha utilizado el framework **React** junto con **TypeScript**, lo que permite construir interfaces de usuario interactivas, robustas y fácilmente escalables. La elección de React responde a su enfoque basado en componentes reutilizables y a la amplia comunidad de soporte, mientras que TypeScript aporta tipado estático, facilitando la detección temprana de errores y el mantenimiento del código a largo plazo.

El diseño visual y la experiencia de usuario se han implementado utilizando la librería de componentes **Material UI (MUI)**, basada en las directrices de Material Design de Google. Esta tecnología proporciona una interfaz moderna, coherente y completamente responsiva, adaptándose automáticamente a dispositivos móviles y de escritorio. Además, se han empleado los breakpoints y utilidades de MUI para garantizar una experiencia óptima en cualquier tamaño de pantalla.

La navegación entre las distintas vistas de la aplicación se gestiona mediante **React Router**, permitiendo una experiencia fluida de usuario sin recargas completas de la página y facilitando la organización de rutas públicas y protegidas. Para la gestión de estados globales, como la autenticación de usuario o el reproductor de música, se ha utilizado la **Context API** de React, lo que permite compartir información entre componentes de manera eficiente y escalable.

La comunicación con el backend se realiza a través de la librería **Axios**, que facilita la realización de peticiones HTTP de forma sencilla y segura. Todo el desarrollo del frontend se ha llevado a cabo en el entorno de **Visual Studio Code**, aprovechando sus herramientas de depuración, integración con sistemas de control de versiones y soporte para TypeScript y React.

El resultado es una aplicación web moderna, modular y mantenible, con una interfaz atractiva y adaptada a las necesidades de los usuarios, que cumple con los estándares actuales de desarrollo frontend.

4.2. Diseño y esquemas de las tecnologías del proyecto

4.2.1. Esquema de despliegue

En este esquema se detalla el entorno de despliegue en la máquina virtual Rocky Linux. Se incluyen los componentes clave como Docker, Kind (Kubernetes in Docker), los volúmenes persistentes, la red interna entre contenedores y las exposiciones de servicios al exterior. Este diseño simula un entorno de producción controlado.

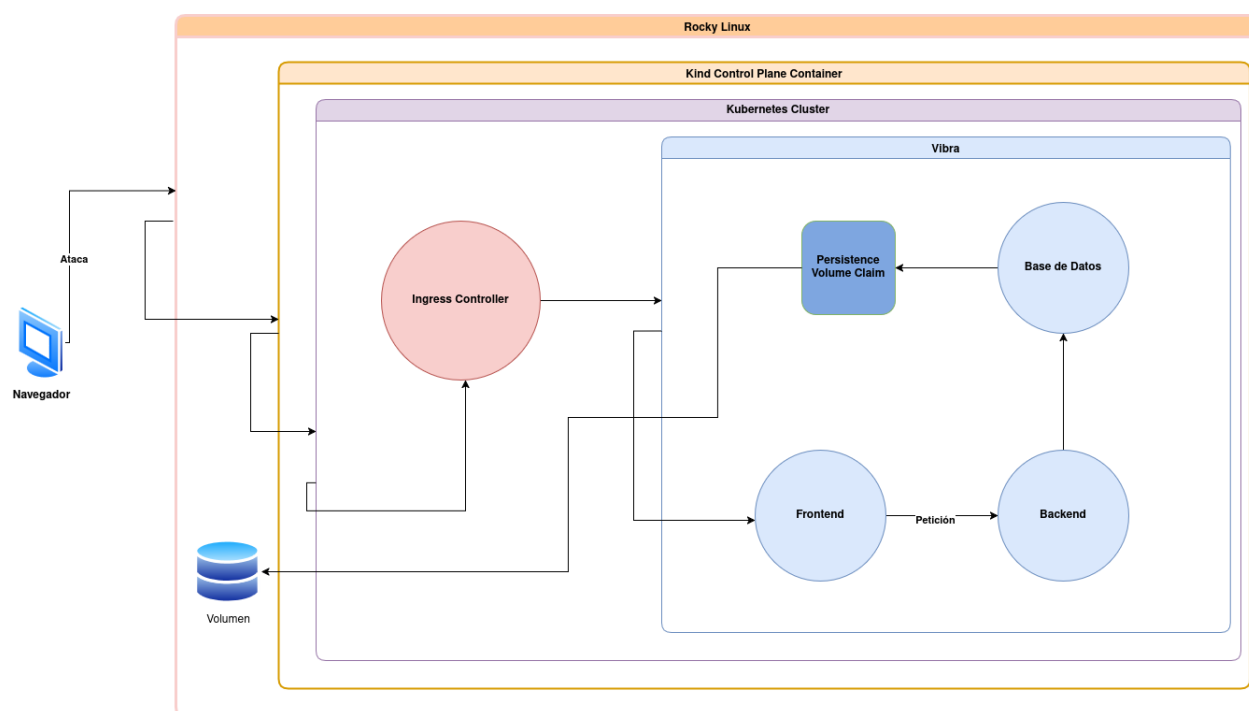


Figura 6: Esquema del entorno de despliegue en el servidor



4.2.2. Diagrama de clases del modelo de datos

En este diagrama se muestra la estructura del modelo de datos del backend. Las clases representan las entidades principales de la aplicación, como User, Post o Album, incluyendo sus atributos y relaciones.

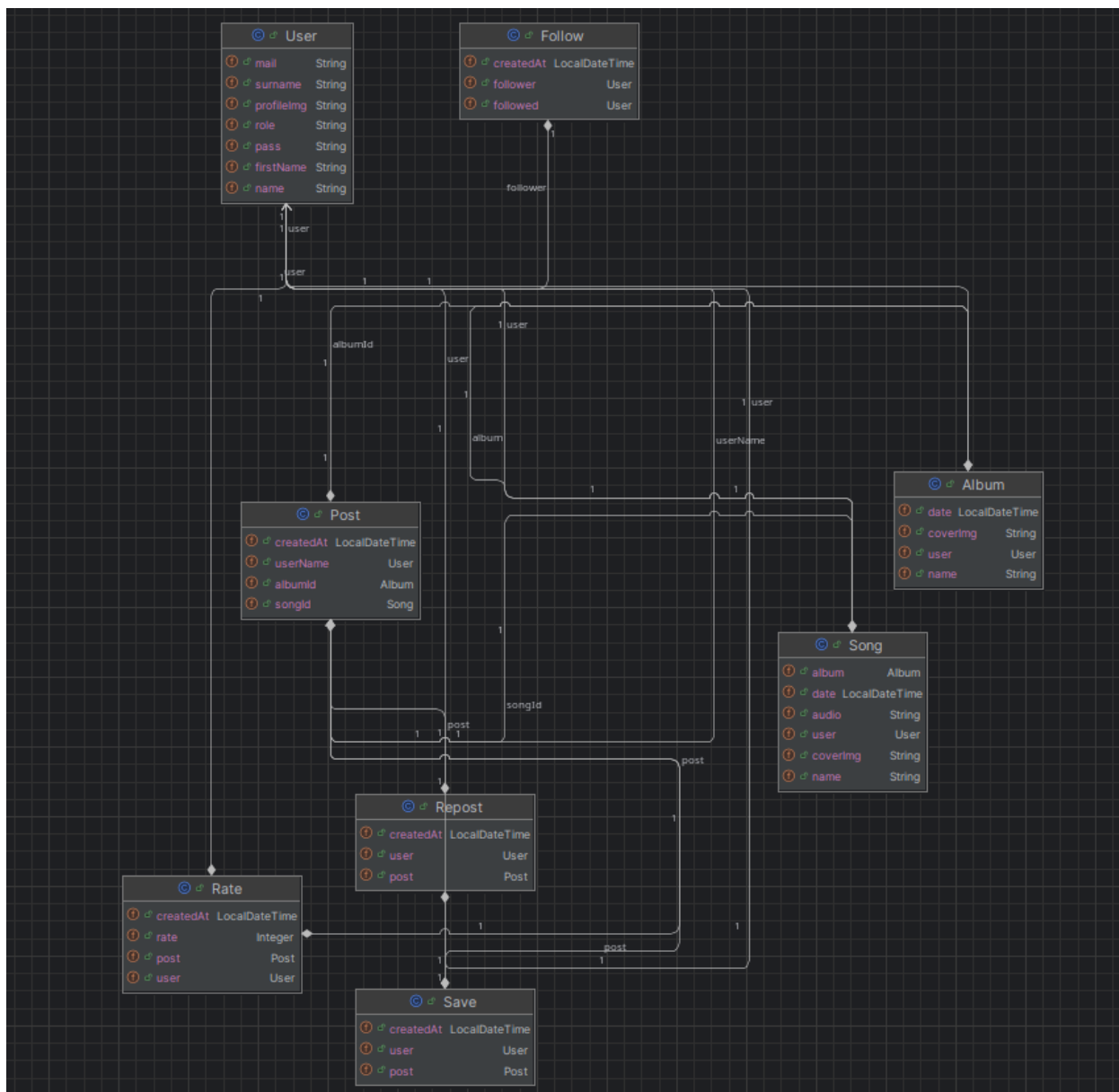


Figura 7: Diagrama de clases del modelo de datos

4.2.3. Esquema de la base de datos

El esquema entidad-relación representa las tablas generadas en la base de datos PostgreSQL a partir de las entidades del backend. Se incluyen las claves primarias, las claves foráneas y los tipos de relaciones entre entidades.

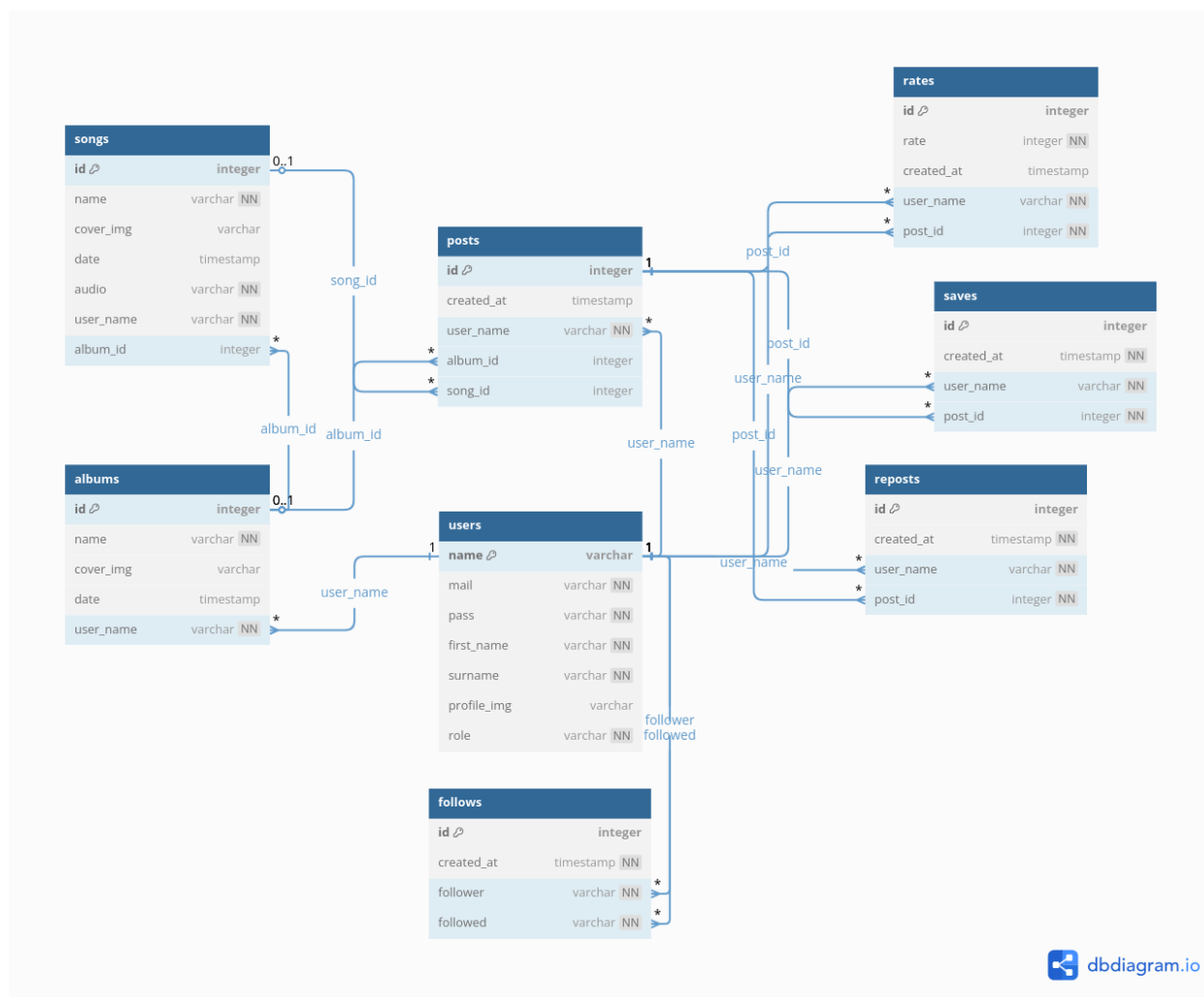


Figura 8: Esquema de la base de datos PostgreSQL

4.2.4. Diagrama de comunicación entre contenedores

Este diagrama representa cómo se comunican los distintos contenedores dentro del clúster de Kubernetes. Se muestran los pods desplegados para el backend, el frontend y la base de datos, así como los servicios que permiten su conexión. Este diseño permite una arquitectura desacoplada, facilitando el mantenimiento y la escalabilidad.

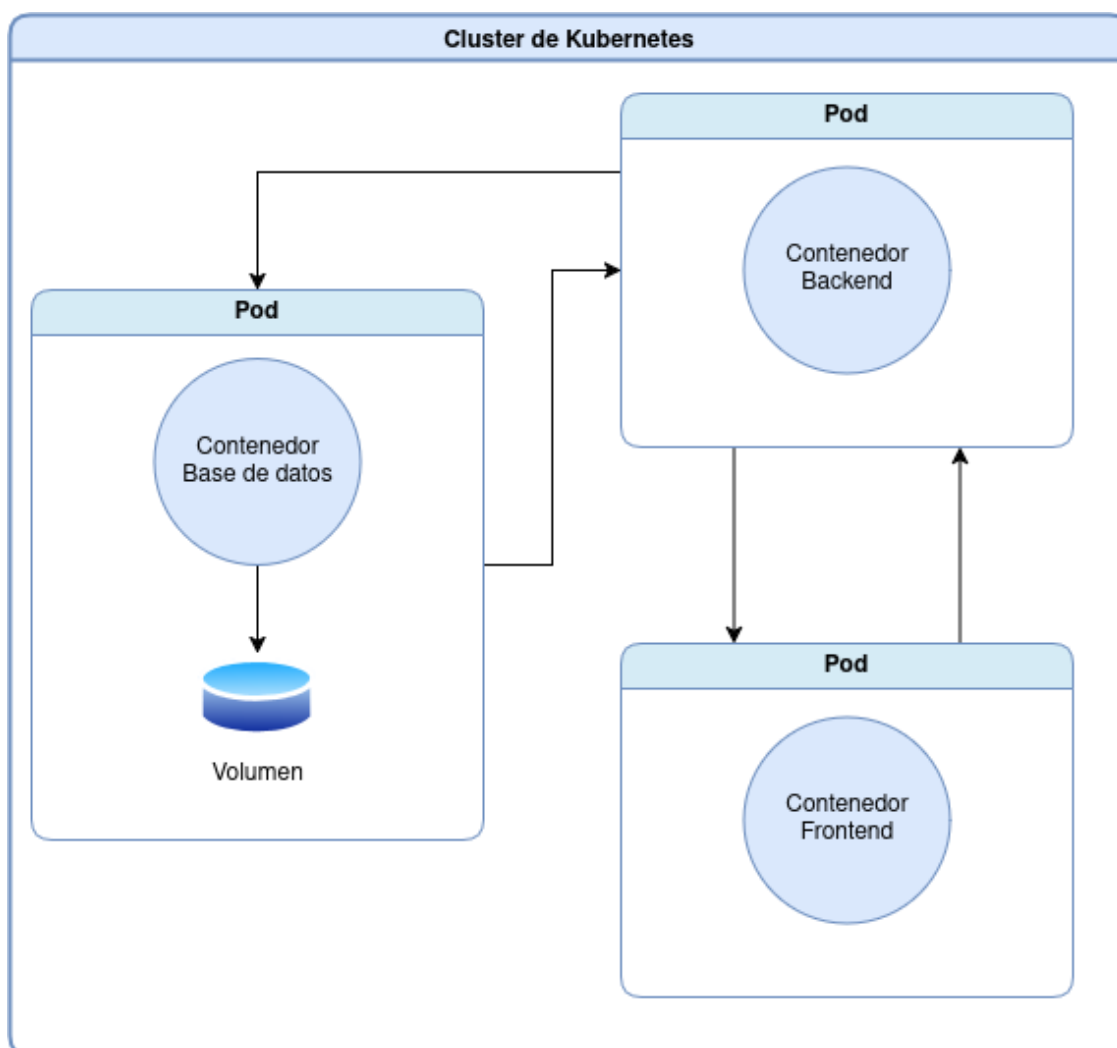


Figura 9: Comunicación entre contenedores en Kubernetes

4.2.5. Diagrama de componentes

El siguiente diagrama representa la estructura de componentes principales de la aplicación frontend desarrollada en React. Se muestra la jerarquía de componentes y la organización modular de la interfaz de usuario, destacando la reutilización de elementos y la separación de responsabilidades.

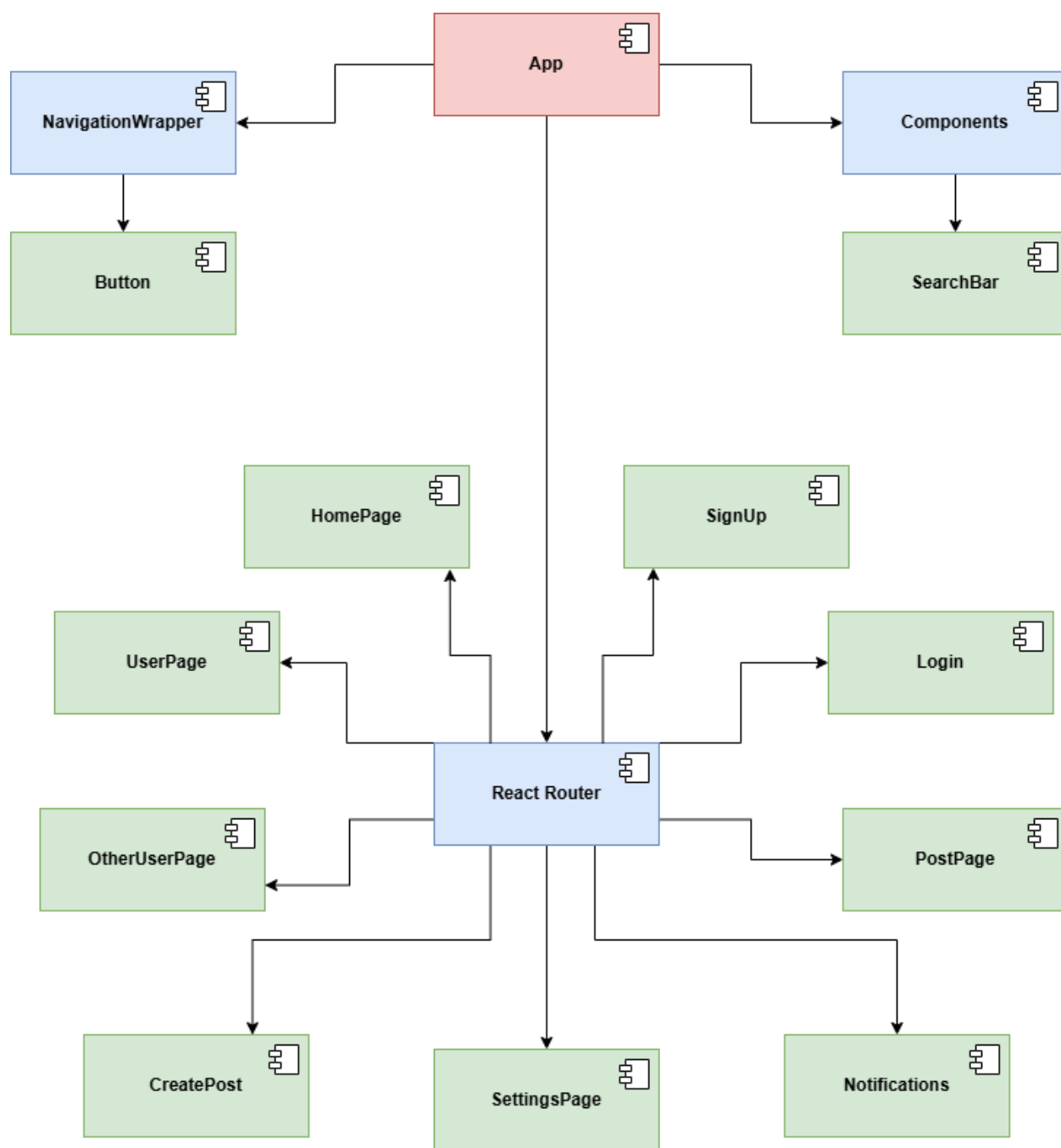


Figura 10: Diagrama de componentes

4.2.6. Diagrama de rutas

El siguiente diagrama ilustra la estructura de rutas principales de la aplicación frontend desarrollada en React. Cada ruta representa una vista o página accesible dentro de la aplicación, gestionada mediante React Router. Se muestran tanto las rutas públicas (como el inicio de sesión o el registro) como las rutas protegidas que requieren autenticación (como el perfil de usuario, la creación de publicaciones o la configuración). Este esquema permite visualizar de forma clara la navegación entre las distintas secciones de la aplicación, facilitando la comprensión de la experiencia de usuario y la organización de las vistas. Además, se reflejan las relaciones y transiciones más relevantes entre las diferentes páginas, lo que contribuye a una arquitectura coherente y mantenible.

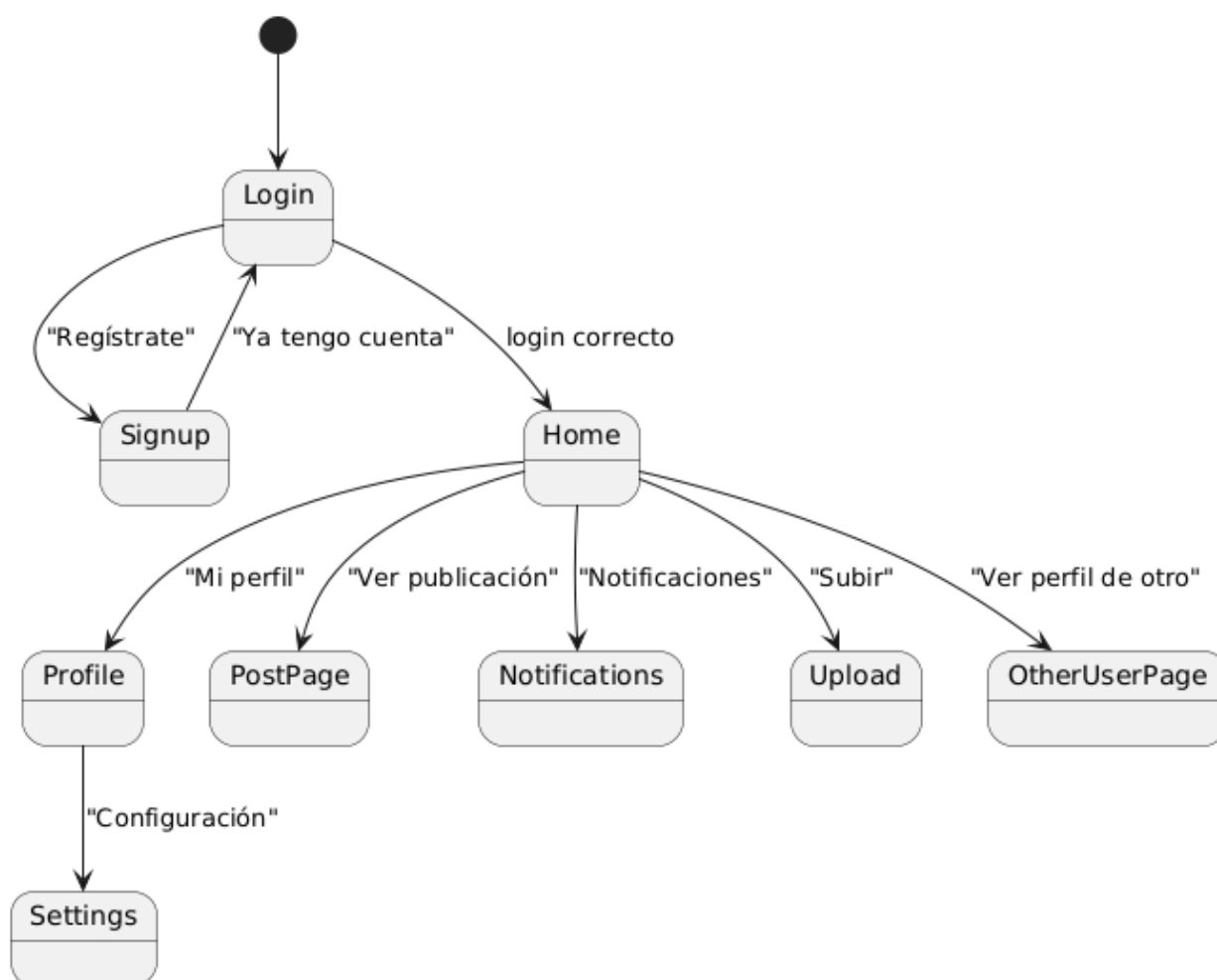


Figura 11: Diagrama de rutas

4.2.7. Diagrama de contextos

El siguiente diagrama representa la arquitectura de gestión de estados globales en el frontend de la aplicación, implementada mediante la Context API de React. En este esquema se muestran los principales proveedores de contexto, como **PlayerContextProvider** y **AuthContextProvider**, que encapsulan y suministran información y funcionalidades esenciales a los distintos componentes y páginas de la aplicación.

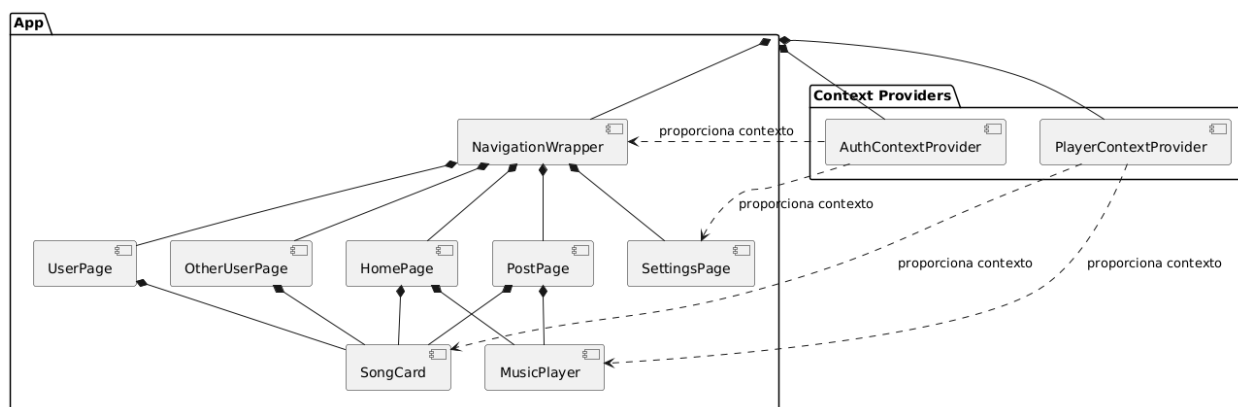


Figura 12: Diagrama de contextos

4.3. Implementación, gestión y configuración del proyecto

4.3.1. Base de datos

La base de datos utilizada en el proyecto es PostgreSQL, elegida por su robustez, fiabilidad y amplio soporte para aplicaciones empresariales. Tras una fase inicial de diseño conceptual y lógico del modelo de datos (que se refleja en el diagrama entidad-relación en el apartado de diseño), se procedió a la implementación física de la base, definiendo tablas, relaciones, índices y restricciones adecuadas para garantizar la integridad y eficiencia del sistema.

Un aspecto especialmente relevante en la implementación ha sido el trabajo con vistas SQL. Estas vistas se diseñaron cuidadosamente para optimizar las consultas realizadas por el backend reactivo, aliviando la carga computacional en la capa de aplicación y permitiendo que ciertas operaciones complejas o combinaciones de tablas se resuelvan directamente en la base de datos. Esto es crucial en un entorno asíncrono y reactivo, donde la eficiencia en la recuperación de datos afecta directamente al rendimiento y la capacidad de respuesta del sistema.

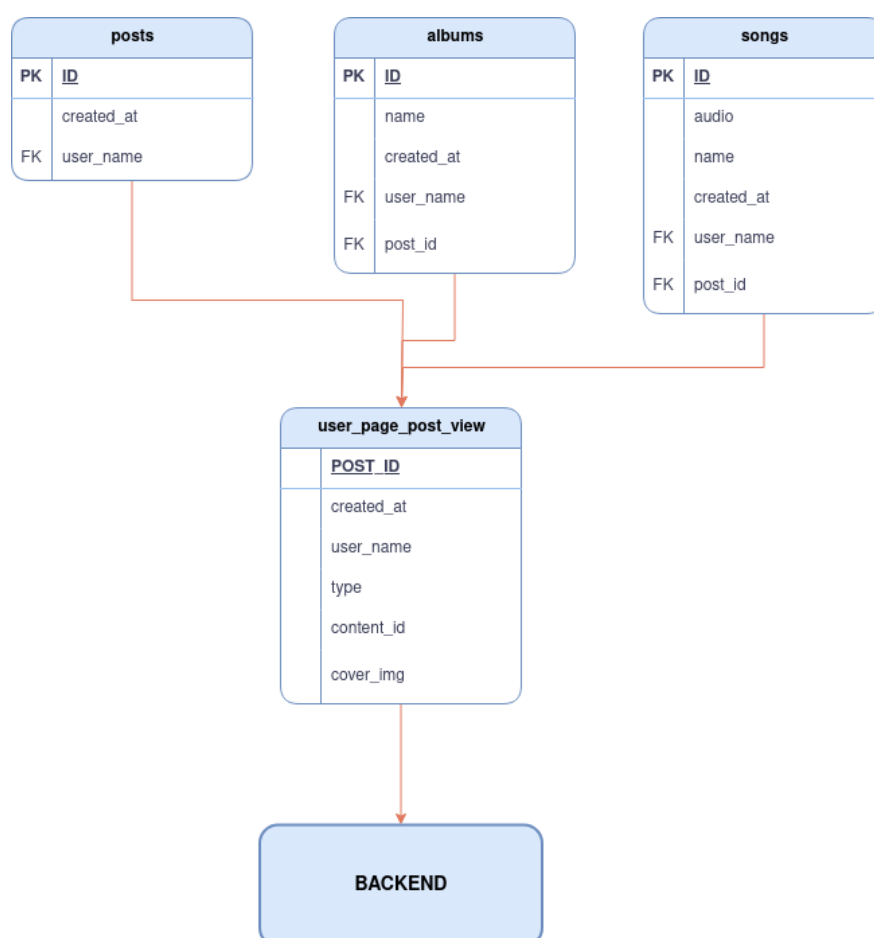


Figura 13: Flujo de una vista



Las vistas permiten, por ejemplo, presentar datos ya preprocesados y combinados, simplificando las consultas emitidas desde Quarkus y facilitando el uso de consultas reactivas sin sacrificar la complejidad funcional necesaria para la aplicación, ya que por causa de una complejidad excesiva el stack de Hibernate podría dar fallos inesperados.

```
1 CREATE OR REPLACE VIEW user_page_posts_view AS
2 SELECT
3     p.id, p.user_name, p.created_at,
4     CASE
5         WHEN p.album_id IS NOT NULL THEN 'album'
6         WHEN p.song_id IS NOT NULL THEN 'song'
7     END AS type,
8     COALESCE(p.album_id, p.song_id) AS content_id,
9     COALESCE(a.name, s.name) AS name,
10    COALESCE(a.cover_img, s.cover_img) AS cover_img
11 FROM posts p
12 LEFT JOIN albums a ON p.album_id = a.id
13 LEFT JOIN songs s ON p.song_id = s.id;
```

Listing 1: Ejemplo de vista

Una vez definido el proyecto de forma completa, se delegó la gestión de la persistencia en Hibernate ORM integrado con Quarkus, utilizando la extensión Reactive Panache. Esto permitió abstraer gran parte de la lógica de acceso a datos, manteniendo la capacidad reactiva y beneficiándose de las optimizaciones que aporta Hibernate para el manejo de entidades y relaciones.

De esta forma, la base de datos y el backend trabajan de forma conjunta para ofrecer una solución escalable y eficiente, en la que la definición clara y precisa del modelo relacional, junto con las vistas optimizadas y el uso de Hibernate reactivo, garantizan una correcta gestión y manipulación de la información.

4.3.2. Backend

Una vez establecidas las tecnologías seleccionadas y definida la arquitectura global del sistema, se procedió a la implementación práctica del backend, abordando su desarrollo desde una perspectiva modular y orientada a la separación clara de responsabilidades. Esta estructura facilita tanto la mantenibilidad del código como su escalabilidad, permitiendo que cada componente pueda evolucionar de forma independiente a medida que la aplicación crezca en funcionalidad.

El backend ha sido desarrollado utilizando el framework **Quarkus**, adoptando un enfoque de **programación reactiva** mediante la extensión **Hibernate Reactive Panache**, lo que permite gestionar de forma no bloqueante las operaciones de acceso a la base de datos. Este modelo asíncrono garantiza un uso eficiente de los recursos del sistema, optimizando el rendimiento bajo cargas concurrentes, y resulta especialmente adecuado para entornos desplegados en contenedores gestionados por Kubernetes.

La implementación comprende múltiples capas funcionales. En primer lugar, se ha realizado el modelado de las entidades de dominio, reflejando la estructura lógica de la base de datos y definiendo las relaciones entre los distintos elementos de la aplicación (usuarios, publicaciones, canciones, álbumes, interacciones, etc.). Sobre estas entidades se ha construido la lógica de negocio encapsulada en los **services**, los cuales interactúan directamente con los modelos de datos y gestionan las operaciones principales de la aplicación.

Por encima de esta capa se encuentran los recursos REST definidos en los **resource**, donde se exponen los distintos **endpoints** de la API que permiten la interacción del frontend con el backend.

Además, se han incorporado distintos mecanismos transversales esenciales para el correcto funcionamiento de la aplicación, como la **autenticación mediante JWT**, la **validación de datos de entrada**, el **manejo centralizado de excepciones** o la **gestión de archivos multimedia**, asegurando en cada caso la seguridad, integridad y eficiencia de la aplicación.

A lo largo de esta sección se verá en detalle la implementación de estas tecnologías, así como ejemplos relevantes que ilustren el funcionamiento y las decisiones técnicas adoptadas durante el desarrollo.

4.3.2.1 Estructuración del código

El código del backend está organizado siguiendo una estructura modular y clara, facilitando la mantenibilidad dentro del proyecto.

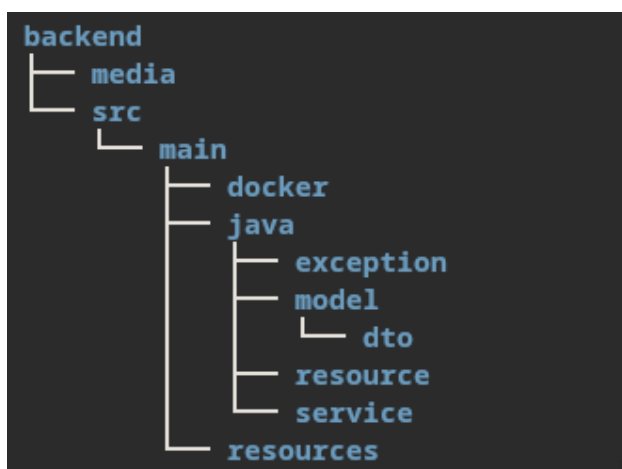


Figura 14: Árbol de directorios del backend

- **media**: directorio destinado a almacenar los archivos multimedia subidos por los usuarios. Estos archivos se guardan de forma física en el servidor y son accesibles posteriormente a través de endpoints específicos del backend.
- **src/main/docker**: contiene los recursos necesarios para la construcción de imágenes.
- **src/main/java**: directorio principal donde reside la lógica del backend. Este paquete se subdivide en los siguientes subpaquetes:
 - **exception**: contiene las clases encargadas de la gestión centralizada de excepciones, permitiendo un control homogéneo de los errores durante la ejecución de la aplicación.
 - **model**: incluye las entidades de dominio que representan las tablas de la base de datos, implementadas mediante `Hibernate Reactive Panache`.
 - **dto**: contiene las clases DTO (Data Transfer Object), utilizadas para estructurar y transportar datos específicos entre el backend y el frontend, simplificando la transferencia de información y evitando exponer las entidades de la base de datos.
 - **resource**: agrupa los controladores REST que definen los endpoints de la API. Cada clase dentro de este paquete expone las operaciones disponibles para los distintos recursos de la aplicación.
 - **service**: contiene la lógica de negocio, encapsulando las operaciones sobre las entidades y gestionando la comunicación entre los `resource` y los `modelos`.
- **src/main/resources**: almacena los archivos de configuración del proyecto.

4.3.2.2 Principales entidades y relaciones

El backend de la aplicación está construido sobre un conjunto de entidades que modelan la información persistida en la base de datos. Estas entidades reflejan los distintos elementos funcionales de la red social y se encuentran implementadas mediante Hibernate Reactive Panache, facilitando así el acceso a los datos de forma reactiva.

A continuación, se describen las entidades principales desarrolladas en el backend y se muestran sus atributos principales:

User

Representa a cada usuario registrado en la aplicación y almacena su información personal y de autenticación. En esta entidad se puede ver el uso de **PanacheEntityBase**, que representa las entidades con un ID gestionado manualmente.

```
1 @Entity
2 @Table(name = "users")
3 public class User extends PanacheEntityBase {
4
5     @Id
6     public String name;
7
8     @Column(length = 100, nullable = false, unique = true)
9     public String mail;
10
11     @Column(length = 100, nullable = false)
12     public String pass;
```

Listing 2: Entidad User

Esta entidad gestiona las relaciones de publicación, seguimiento e interacciones sociales dentro de la plataforma, además centraliza todas las acciones de autenticación y obtención de datos mediante Json Web Token.



Post

Representa una publicación creada por un usuario. Cada publicación está vinculada a un contenido musical, pudiendo ser una canción o un álbum, pero nunca ambos simultáneamente.

```
1 Entity
2 @Table(name = "posts")
3 public class Post extends PanacheEntity {
4
5     @Column(name = "created_at", nullable = false)
6     public LocalDateTime createdAt;
7
8     @ManyToOne
9     @JoinColumn(name = "user_name", nullable = false)
10    public User userName;
11
12    @ManyToOne
13    @JoinColumn(name = "album_id", nullable = true)
14    public Album albumId;
15
16    @ManyToOne
17    @JoinColumn(name = "song_id", nullable = true)
18    public Song songId;
```

Listing 3: Entidad Post

Se puede ver también cómo se gestionan las claves foráneas, además de la clase **PanacheEntity** que gestiona de forma automática el ID de una entidad.

Song

Representa una canción individual que puede formar parte de un álbum y ser utilizada en las publicaciones de los usuarios. Cada canción puede estar o no relacionada con un álbum. En el caso de estarlo, formará parte del mismo.

```
1 @Entity
2 @Table(name = "songs")
3 public class Song extends PanacheEntity {
4
5     @Column(length = 100, nullable = false)
6     public String name;
7
8     @Column(nullable = false)
9     public String audio;
10
11    @ManyToOne
12    @JoinColumn(name = "album_id", nullable = true)
13    public Album album;
```

Listing 4: Entidad Song



Álbum

Representa un álbum musical. En esta entidad se puede ver cómo se gestionan las claves foráneas con **quarkus-rest-jackson** cuando no se quieren mostrar en su formato Json.

```
1 @Entity
2 @Table(name = "albums")
3 public class Album extends PanacheEntity {
4
5     @Column(length = 100, nullable = false)
6     public String name;
7
8     @Column(name = "cover_img")
9     public String coverImg;
10
11     @ManyToOne
12     @JoinColumn(name = "user_name", nullable = false)
13     public User user;
14
15     @JsonProperty("user")
16     public String getJsonUser() {
17         return user.name;
18     }
19
20     @JsonIgnore
21     public User getUser() {
22         return user;
23     }
24 }
```

Listing 5: Entidad Album

Follow

Gestiona las relaciones de seguimiento entre usuarios, permitiendo construir la red social de interacciones. Se puede ver cómo contiene 2 claves foráneas pertenecientes a la misma entidad.

```
1 @Entity
2 @Table(name = "follows")
3 public class Follow extends PanacheEntity {
4
5     @ManyToOne
6     @JoinColumn(name = "follower", nullable = false)
7     public User follower;
8
9     @ManyToOne
10    @JoinColumn(name = "followed", nullable = false)
11    public User followed;
12 }
```

Listing 6: Entidad Follow



Repost

Representa la acción de republicación por parte del usuario.

```
1 @Entity
2 @Table(name = "reposts")
3 public class Repost extends PanacheEntity {
4
5     @ManyToOne
6     @JoinColumn(name = "user_name", nullable = false)
7     public User user;
8
9     @ManyToOne
10    @JoinColumn(name = "post_id", nullable = false)
11    public Post post;
```

Listing 7: Entidad Repost

Rate y Save

Representa las calificaciones y guardados que los usuarios asignan a las publicaciones de otros usuarios, respectivamente. Estas 2 entidades son muy similares, por lo que pondré 1 como ejemplo.

```
1 @Entity
2 @Table(name = "rates")
3 public class Rate extends PanacheEntity {
4
5     @Column(nullable = false)
6     public Integer rate;
7
8     @ManyToOne
9     @JoinColumn(name = "user_name", nullable = false)
10    public User user;
11
12    @ManyToOne
13    @JoinColumn(name = "post_id", nullable = false)
14    public Post post;
```

Listing 8: Entidad Rate

4.3.2.3 Lógica de negocio y programación asíncrona

Hasta este punto se ha descrito el modelo reactivo general adoptado en el backend y su integración con Quarkus. En este apartado se profundiza en la aplicación concreta de la programación asíncrona en la lógica de negocio.

Programación asíncrona con Mutiny

Tal como se explicó anteriormente, el backend está diseñado bajo un modelo reactivo, evitando operaciones bloqueantes para optimizar el uso de recursos y soportar grandes volúmenes de conexiones simultáneas. Para implementar esta lógica asíncrona se ha utilizado la librería Mutiny, integrada de forma nativa en Quarkus.

Mutiny permite trabajar sobre los tipos `Uni<T>` y `Multi<T>`, los cuales representan flujos de datos asíncronos. En el caso de este proyecto, la mayoría de operaciones sobre la base de datos devuelven instancias de `Uni<T>`, dado que normalmente se trabaja con una única respuesta por consulta.

Este modelo permite encadenar operaciones sin necesidad de bloquear el hilo de ejecución. Las operaciones se definen mediante métodos como `onItem()`, `flatMap()` o `onFailure()`, estableciendo de forma declarativa el flujo de transformación y manejo de errores.

A continuación, se muestra un ejemplo simple de cómo recuperar un usuario por su nombre de forma reactiva:

```
1 public Uni<User> getUserByName(String name) {  
2     return User.<User>find("name", name).firstResult()  
3         .onItem().ifNull()  
4         .failWith(new CustomNotFoundException("User not found: " +  
5             + name));  
6 }
```

Listing 9: Consulta simple con Uni



En los casos donde es necesario realizar múltiples operaciones secuenciales, Mutiny permite encadenar las mismas de forma clara y no bloqueante. A continuación, se muestra un ejemplo en el que se encadenan consultas para obtener datos adicionales tras recuperar un usuario:

```
1 @WithTransaction
2 public Uni<Response> deleteSave(SecurityIdentity si, Long post) {
3     return us.getUserByToken(si)
4         .flatMap(user ->
5             Save.delete("post.id=?1 AND user=?2", post, user)
6             .onItem().transform(ignore ->
7                 Response.status(204).build())
8         );
9 }
```

Listing 10: Encadenamiento de operaciones

Como se puede ver en el ejemplo, se utiliza `flatMap()` para obtener el usuario devuelto por la consulta anterior y tratarlo como un objeto simple que es utilizado para persistir un `Save` y devolver un código de estado 204 (created).

También es destacable mencionar el uso de `SecurityIdentity` para la obtención del usuario, esto se verá en detalle mas adelante.



Lógica de negocio

La lógica de negocio de la aplicación está implementada en la capa de servicios, donde se encapsulan las operaciones principales que gestionan el flujo de datos entre la API REST y la base de datos. A continuación, se presentan dos ejemplos que ilustran la aplicación práctica de la programación reactiva en distintos niveles de complejidad.

Ejemplo simple: creación de una publicación

En primer lugar, se muestra un ejemplo de creación de una publicación, donde se ilustra el uso de Uni para realizar operaciones encadenadas de persistencia. Dado que el backend está diseñado para optimizar la carga de peticiones, es necesario realizar múltiples operaciones de persistencia en una única transacción lógica. Esto se debe a la complejidad del modelo de datos, ya que al crear una publicación de tipo álbum, deben insertarse simultáneamente, además del post tanto los datos del álbum como las canciones asociadas.

```
1 public Uni<Album> createAlbumfromPost(User user, AlbumPostDTO postDTO) {
2     Album a = new Album(postDTO.albumName, postDTO.coverImg,
3         LocalDateTime.now(), user);
4     return persistAlbum(a)
5         .flatMap(album -> {
6             return Multi.createFrom().iterable(postDTO.songs)
7                 .onItem().transformToUniAndMerge(song -> {
8                     Song s = new Song(
9                         song.name,
10                        postDTO.coverImg,
11                        LocalDateTime.now(),
12                        song.audio,
13                        user,
14                        album
15                    );
16                    return ss.persistSong(s);
17                }).collect().asList()
18                .replaceWith(album);
19 });
}
```

Listing 11: Creación de álbum



```
1 @WithTransaction
2 public Uni<Response> addAlbumPost(SecurityIdentity si, AlbumPostDTO
   postDTO) {
3     return us.getUserByToken(si)
4         .flatMap(user -> {
5         return as.createAlbumfromPost(user, postDTO)
6             .flatMap(album -> {
7                 Post post = new Post(
8                     LocalDateTime.now(),
9                     user,
10                    album,
11                    null
12                );
13                return persistPost(post);
14            });
15     });
16 }
```

Listing 12: Creación de publicación

Véase como se encadena de forma asíncrona el método de `createAlbumFromPost()` con la creación y persistencia del propio post.



Ejemplo complejo: generación del feed Home

A continuación, se presenta un ejemplo más representativo de la lógica de negocio real implementada en el proyecto. Este método genera el contenido del *feed* de inicio, combinando publicaciones y reposts recientes de los usuarios seguidos, ordenados cronológicamente. Se puede observar el uso intensivo de composición reactiva mediante Mutiny para gestionar múltiples operaciones dependientes de forma asíncrona.

Es importante aclarar que por no extender de más el documento se ha escogido 1 solo de los 6 métodos que componen esta operación.

```
1 public Uni<List<HomeDTO>> getHomeObjects(List<User> followed, Integer
2     page, Integer pageSize) {
3     if (followed.isEmpty()) {
4         return Uni.createFrom().item(List.of());
5     }
6     return ps.getLatestPostsOf(followed, page, pageSize)
7         .onFailure().invoke(e -> System.err.println("Error en
8             getLatestPostsOf: " + e))
9         .flatMap(posts -> Multi.createFrom().iterable(posts)
10            .onItem().<HomeDTO>transformToUniAndMerge(post -> {
11                if (post.albumId != null) {
12                    return ss.getSongsByAlbum(post.albumId.id)
13                        .flatMap(songs -> homeOfPost(post, songs));
14                }
15                return homeOfPost(post, null);
16            })
17            .collect().asList()
18            .flatMap(homePostDTOs -> rs.getLatestRepostsOf(followed, page,
19                pageSize)
20                .flatMap(reposts -> Multi.createFrom().iterable(reposts)
21                    .onItem().<HomeDTO>transformToUniAndMerge(repost -> {
22                        if (repost.post.albumId != null) {
23                            return ss.getSongsByAlbum(repost.post.albumId.id)
24                                .flatMap(songs -> homeOfRepost(repost, songs));
25                        }
26                        return homeOfRepost(repost, null);
27                    })
28                    .collect().asList()
29                    .map(homeRepostDTOs -> {
30                        List<HomeDTO> items = new ArrayList<>();
31                        items.addAll(homePostDTOs);
32                        items.addAll(homeRepostDTOs);
33                        return items.stream()
34                            .sorted((a, b) -> b.createdAt.compareTo(a.createdAt))
35                            .toList();
36                    })))));
37 }
```

Listing 13: Generación del feed Home



Explicación del flujo de operaciones:

- A la función se le pasa una lista de usuarios que sigue el usuario autenticado obtenida previamente y el número y tamaño de página de la consulta a base de datos.
- En primer lugar, se verifica si la lista de usuarios seguidos está vacía (En ese caso, se retorna directamente una lista vacía).
- Si existen usuarios seguidos, se obtienen sus publicaciones recientes mediante una llamada al servicio de publicaciones.
- Por cada publicación recuperada, se transforma el resultado a su correspondiente DTO:
 - Si la publicación está asociada a un álbum, se consultan las canciones de dicho álbum.
 - Si está asociada a una canción individual, se procesa directamente.
- Posteriormente, se recuperan los reposts recientes de los mismos usuarios.
- Cada repost se procesa de forma similar, transformándolo a su correspondiente DTO.
- Finalmente, se combinan ambos resultados (posts y reposts), se ordenan cronológicamente y se devuelven como lista final.



4.3.2.4 Autenticación con JWT

La autenticación del backend se ha implementado mediante el estándar JWT (JSON Web Token), aprovechando la extensión nativa de Quarkus `quarkus-smallrye-jwt`. Este sistema permite mantener una arquitectura sin estado (stateless), donde la información de autenticación viaja en cada petición, evitando la necesidad de sesiones persistentes.

El token es generado en el backend cuando un usuario inicia sesión de forma correcta. Para ello se ha implementado un servicio específico encargado de la emisión de tokens firmado digitalmente:

```
1 public class JWTService {
2
3     public static String generateToken(String userName, String role) {
4         return Jwt
5             .issuer("https://vibra.com")
6             .subject(userName)
7             .groups(role)
8             .expiresIn(Duration.ofDays(10))
9             .sign();
10    }
11 }
```

Listing 14: Servicio de generación de JWT

El sistema utiliza un par de claves asimétricas (pública y privada) en formato PEM para firmar y verificar los tokens. Estas claves están almacenadas como Secrets dentro del *Pod* de Kubernetes donde se despliega el backend, y se inyectan como variables de entorno al contenedor a través de la configuración YAML del despliegue.

```
1 env:
2 - name: MP_JWT_VERIFY_PUBLICKEY
3   valueFrom:
4     secretKeyRef:
5       name: jwt-secret
6       key: publicKey.pem
7 - name: SMALLRYE_JWT_SIGN_KEY
8   valueFrom:
9     secretKeyRef:
10      name: jwt-secret
11      key: privateKey.pem
```

Listing 15: Configuración del Secret y variables de entorno



Una vez emitido el token, el frontend lo incluye en cada petición HTTP mediante la cabecera `Authorization: Bearer <token>`. El backend valida automáticamente el token recibido utilizando el mecanismo de seguridad de Quarkus.

Durante la ejecución, el contexto de seguridad de Quarkus expone el `SecurityIdentity`, que permite acceder a los datos del usuario autenticado directamente desde cualquier punto del backend. A continuación se muestra un ejemplo de cómo acceder al usuario autenticado:

```
1 public Uni<User> getUserByToken(SecurityIdentity securityIdentity) {  
2     String userName =  
3         securityIdentity.getPrincipal().getName();  
4     return getUserByName(userName);  
}
```

Listing 16: Acceso al usuario autenticado con `SecurityIdentity`



4.3.2.5 Gestión de archivos multimedia

El backend permite la subida de archivos multimedia por parte de los usuarios, principalmente imágenes y audios, que posteriormente son referenciados desde las publicaciones. La subida de archivos se realiza mediante peticiones HTTP POST con formularios de tipo multipart/form-data.

Al recibir un archivo, el backend valida su tipo MIME utilizando la biblioteca Apache Tika con el objetivo de garantizar que solo se acepten los formatos permitidos, mitigando así riesgos de seguridad relacionados con el procesamiento de ficheros maliciosos.

A continuación se muestra un ejemplo de la lógica implementada para la subida de archivos:

```
1 @POST
2 @jakarta.ws.rs.Path("/upload")
3 public Response uploadFile(@BeanParam FileUploadFormDTO form) {
4     try {
5         Path uploadedPath = form.file.uploadedFile();
6         String mimeType = tika.detect(uploadedPath);
7
8         if (!ALLOWED_TYPES.contains(mimeType)) {
9             return Response.status(Response.Status.BAD_REQUEST)
10                .entity(Map.of("error", "Tipo de archivo no
11                               permitido: " + mimeType))
12                .build();
13         }
14
15         String extension = getExtension(form.file.fileName());
16         String uuid = UUID.randomUUID().toString();
17         String safeFileName = uuid + extension;
18         Path targetPath = Path.of(UPLOAD_DIR, safeFileName);
19
20         Files.createDirectories(targetPath.getParent());
21         Files.move(uploadedPath, targetPath,
22             StandardCopyOption.REPLACE_EXISTING);
23
24         return Response.ok(Map.of("url", URL + safeFileName)).build();
25     } catch (Exception e) {
26         return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
27            .entity(Map.of("error", e.getMessage()))
28            .build();
29     }
30 }
```

Listing 17: Subida de archivos y validación

Los archivos aceptados son almacenados de forma local dentro del servidor, en un directorio dedicado para este fin. Gracias a la validación previa de los tipos de archivo mediante Tika, se garantiza que solo se acepten contenidos válidos.

4.3.2.6 Endpoints de la Api

La API del proyecto se estructura en torno a múltiples módulos funcionales que permiten gestionar la experiencia de usuario dentro de la red social. Cada grupo de endpoints está orientado a un recurso específico del sistema, y todos están diseñados para funcionar de manera reactiva, permitiendo alta concurrencia y eficiencia. Además, todos los endpoint funcionan sobre el usuario autenticado mediante JWT.

A continuación, se describen los principales grupos de endpoints:

- **Home:** Permite obtener la información personalizada del usuario (posts, canciones, álbumes), así como consultar métricas asociadas a las publicaciones. También incluye un buscador general.
- **Users:** Incluye operaciones de registro, autenticación (login), obtención de datos del perfil y actualizaciones. Se ofrecen endpoints tanto para el usuario autenticado como para consultar la información de otros usuarios.
- **Posts:** Gestiona la creación, consulta y eliminación de publicaciones de tipo canción o álbum.
- **Metrics:** Permite puntuar, guardar o hacer repost de publicaciones, así como eliminar estas acciones.
- **Songs:** Facilita el acceso a la información detallada de canciones y álbumes.
- **Follows:** Gestiona las relaciones de seguimiento entre usuarios.
- **Notifications:** Proporciona la lista de notificaciones recibidas por el usuario autenticado.
- **Media:** Permite subir uno o varios archivos multimedia, devolviendo URLs públicas para su visualización o reproducción.



Método	Ruta	Descripción
GET	/api/home	Obtener objetos del feed Home
GET	/api/home/post_id	Obtener métricas de una publicación
GET	/api/home/search/search	Buscador de usuarios, álbumes y canciones
POST	/api/metrics/rate	Puntuar una publicación
PUT	/api/metrics/rate	Actualizar puntuación de una publicación
POST	/api/metrics/save	Guardar una publicación
DELETE	/api/metrics/save/post_id	Eliminar guardado de una publicación
POST	/api/metrics/repost	Repostear una publicación
DELETE	/api/metrics/repost/post_id	Eliminar repost de una publicación
POST	/api/users/login	Login (autenticación)
POST	/api/users/register	Registro de usuario
GET	/api/users/page	Obtener datos del usuario autenticado
GET	/api/users/page/user_name	Obtener datos de otro usuario
GET	/api/users/posts	Obtener publicaciones del usuario
GET	/api/users/posts/user_name	Obtener publicaciones de otro usuario
GET	/api/users/rates	Obtener publicaciones puntuadas
GET	/api/users/saves	Obtener publicaciones guardadas
PATCH	/api/users/update/field/value	Actualizar campo de usuario
PATCH	/api/users/update/password	Actualizar contraseña del usuario
DELETE	/api/users/user_name	Eliminar cuenta de usuario
GET	/api/follows/followed	Obtener lista de usuarios seguidos
GET	/api/follows/followed/user_name	Obtener lista de usuarios seguidos
GET	/api/follows/followers	Obtener lista de seguidores
GET	/api/follows/followers/user_name	Obtener lista de seguidores
GET	/api/follows/follow/user_name	Comprobar si sigue a un usuario
POST	/api/follows/follow/user_name	Seguir a un usuario
DELETE	/api/follows/follow/user_name	Dejar de seguir a un usuario
GET	/api/songs/id	Obtener canción por ID
GET	/api/songs/albums/album_id	Obtener canciones por álbum
POST	/api/posts/song	Crear publicación con canción
POST	/api/posts/album	Crear publicación con álbum
GET	/api/posts/post_id	Obtener información de una publicación
DELETE	/api/posts/post_id	Eliminar publicación
POST	/api/media/upload	Subir un archivo multimedia
POST	/api/media/upload/multi	Subir múltiples archivos multimedia
GET	/api/notifications	Obtener notificaciones
GET	/api/users/reposts	Obtener publicaciones repostadas
GET	/api/users/reposts/user_name	Obtener publicaciones reposteadas

Cuadro 2: Listado completo de endpoints del backend

4.3.3. Servidor y despliegue

Para el despliegue del proyecto se ha creado un entorno controlado de servidor, simulado de forma realista para reflejar un sistema de producción funcional. A continuación se describe el proceso completo de configuración del entorno de despliegue.

4.3.3.1 Sistema operativo: Rocky Linux

Como base del servidor se ha optado por la distribución **Rocky Linux**, una alternativa estable, ligera y compatible con entornos empresariales basada en Red Hat Enterprise Linux (RHEL). Esta elección responde a varios criterios:

- Proporciona un entorno robusto, mantenido a largo plazo y con actualizaciones de seguridad continuas.
- Permite una instalación mínima, reduciendo la presencia de servicios innecesarios y optimizando los recursos del servidor.
- Es ampliamente compatible con herramientas de contenedores, orquestación y sistemas cloud.

Una vez desplegado el sistema, se procedió a la apertura de los puertos necesarios para el correcto funcionamiento de los servicios, incluyendo los puertos de Kubernetes, SSH, HTTP y HTTPS.

```
1 $> systemctl list-units --type=service --state=running
2
3 $> sudo ss -tln
4
5 $> sudo firewall-cmd --permanent --add-port=80/tcp
6 $> sudo firewall-cmd --permanent --add-port=443/tcp
7 $> sudo firewall-cmd --permanent --add-port=22/tcp
8 $> sudo firewall-cmd --permanent --add-port=6443/tcp
```

Listing 18: Apertura de puertos

4.3.3.2 Creación del clúster Kubernetes con Kind

Para gestionar la orquestación de contenedores, se configuró un clúster local de Kubernetes mediante la herramienta **Kind (Kubernetes IN Docker)**. Esta herramienta permite crear clústeres de Kubernetes dentro de contenedores, facilitando tanto el desarrollo como el testeo del entorno completo en la propia máquina del servidor.

La instalación de Kind es sencilla y está ampliamente documentada en su guía oficial. A continuación se incluyen los comandos empleados para su instalación y la creación del clúster:

```
1 $> cat <<EOF | kind create cluster --name vibra --config=-
2 kind: Cluster
3 apiVersion: kind.x-k8s.io/v1alpha4
4 nodes:
5 - role: control-plane
6   extraPortMappings:
7   - containerPort: 80
8     hostPort: 80
9     protocol: TCP
10  - containerPort: 443
11    hostPort: 443
12    protocol: TCP
13  - containerPort: 8080
14    hostPort: 8080
15    protocol: TCP
16 EOF
17
18 $> k apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-
19   ingress-nginx.yaml
20
21 $> k wait --namespace ingress-nginx \
22   --for=condition=ready pod \
23   --selector=app.kubernetes.io/component=controller \
24   --timeout=90s
```

Listing 19: Instalación de Kind y creación de clúster

4.3.3.3 Creación de contenedores: Dockerfiles y configuración

Una vez configurado el clúster, se procedió a construir las imágenes necesarias para cada uno de los servicios del proyecto. Se generaron los contenedores a partir de los correspondientes Dockerfiles, adaptados a los requerimientos específicos de cada componente:

- **Base de datos (PostgreSQL):** Imagen oficial de PostgreSQL sin necesidad de configuración adicional.
- **Backend (Quarkus):** Imagen generada a partir de la aplicación compilada en modo JVM, optimizando el tamaño y el rendimiento de la imagen.

```
1 FROM registry.access.redhat.com/ubi9/openjdk-21:1.21
2
3 ENV LANGUAGE='en_US:en'
4
5
6 # We make four distinct layers so if there are application
   changes the library layers can be re-used
7 COPY --chown=185 target/quarkus-app/lib/ /deployments/lib/
8 COPY --chown=185 target/quarkus-app/*.jar /deployments/
9 COPY --chown=185 target/quarkus-app/app/ /deployments/app/
10 COPY --chown=185 target/quarkus-app/quarkus/ /deployments/
   quarkus/
11
12 EXPOSE 8080
13 USER 185
14 ENV JAVA_OPTS_APPEND="-Dquarkus.http.host=0.0.0.0 -Djava.util.
   logging.manager=org.jboss.logmanager.LogManager"
15 ENV JAVA_APP_JAR="/deployments/quarkus-run.jar"
16
17 ENTRYPOINT [ "/opt/jboss/container/java/run/run-java.sh" ]
```

Listing 20: Dockerfile del backend



- **Frontend (React):** Imagen basada en Nginx. Fue necesario adaptar la configuración de Nginx para servir correctamente el frontend y gestionar el enrutamiento interno de la aplicación.

```
1 FROM node:20 AS builder
2
3 WORKDIR .
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9 RUN npm run build
10
11 FROM docker.io/library/nginx
12
13 COPY --from=builder /dist /usr/share/nginx/html
14
15 EXPOSE 80
16
17 CMD ["nginx", "-g", "daemon off;"]
```

Listing 21: Dockerfile del frontend

```
1 server {
2
3     root /usr/share/nginx/html;
4     index index.html;
5
6     location / {
7         try_files $uri $uri/ /index.html;
8     }
9
10 }
```

Listing 22: Configuración de Nginx

Las imágenes se subieron a la plataforma [Quay.io](https://quay.io) para evitar traspaso de archivos constante y así el servidor únicamente tiene que utilizarlas.



4.3.3.4 Creación de recursos de Kubernetes

Una vez construidas las imágenes, se definieron los distintos recursos de Kubernetes mediante archivos de configuración en formato YAML. La infraestructura desplegada queda organizada en los siguientes bloques:

Base de datos

- **Deployment:** Define el pod de PostgreSQL.
- **PersistentVolumeClaim (PVC):** Permite la persistencia de datos.
- **Service:** Exponer la base de datos internamente.

Backend

- **Deployment:** Define el pod del backend de Quarkus.
- **Service:** Exponer el backend al frontend e ingress.

Frontend

- **Deployment:** Contenedor de React servido por Nginx.
- **Service:** Exponer el frontend dentro del clúster.
- **Ingress:** Definición de rutas de entrada al clúster.



A continuación se verán 2 ejemplos de estos recursos de Kubernetes en formato YAML:

```
1 kind: Deployment
2 metadata:
3   name: backend
4 spec:
5   replicas: 1
6   selector:
7     matchLabels:
8       app: backend
9   template:
10    metadata:
11      labels:
12        app: backend
13    spec:
14      containers:
15        - name: backend
16          image: quay.io/nicobuquet/vibra:0.3
17          ports:
18            - containerPort: 8080
19          env:
20            - name: MP_JWT_VERIFY_PUBLICKEY
21              valueFrom:
22                secretKeyRef:
23                  name: jwt-secret
24                  key: publicKey.pem
25            - name: SMALLRYE_JWT_SIGN_KEY
26              valueFrom:
27                secretKeyRef:
28                  name: jwt-secret
29                  key: privateKey.pem
30            - name: QUARKUS_HTTP_LIMITS_MAX-BODY-SIZE
31              value: 50M
32            - name: MEDIA_PATH
33              value: http://vibra/api/media/
34            - name: QUARKUS_HIBERNATE-ORM_DATABASE_GENERATION
35              value: drop-and-create
```

Listing 23: Deployment del backend

Como se puede observar en la imagen, el deployment contiene la información de creación del pod, como imagen, puertos y variables de entorno.



```
1 kind: Service
2 metadata:
3   name: db-service
4 spec:
5   selector:
6     app: db-vibra
7   ports:
8     - protocol: TCP
9       port: 5432
10       targetPort: 5432
11   type: ClusterIP
```

Listing 24: Service de la base de datos

Los service contienen información de red. Son los encargados de conectar los pods entre si.

4.3.4. Frontend

4.3.4.1 Configuración del entorno

Como entorno de desarrollo se ha utilizado Visual Studio Code, aprovechando sus extensiones para React, TypeScript, control de versiones y depuración. Visual Studio Code facilita la edición colaborativa, la navegación por el proyecto y la integración con herramientas externas como GitHub.

Para el desarrollo del frontend se ha utilizado Node.js como entorno de ejecución de JavaScript. La instalación se realiza descargando la versión recomendada desde la web oficial y siguiendo el instalador correspondiente al sistema operativo.

Una vez instalado Node.js, se inicializa el proyecto y se gestionan las dependencias mediante npm (Node Package Manager), aunque también sería posible utilizar yarn como alternativa. Para instalar todas las dependencias necesarias, basta con ejecutar el comando `npm install`.

4.3.4.2 Estructura de carpetas

El proyecto frontend está organizado en una estructura de carpetas clara y modular, facilitando la escalabilidad y el mantenimiento.

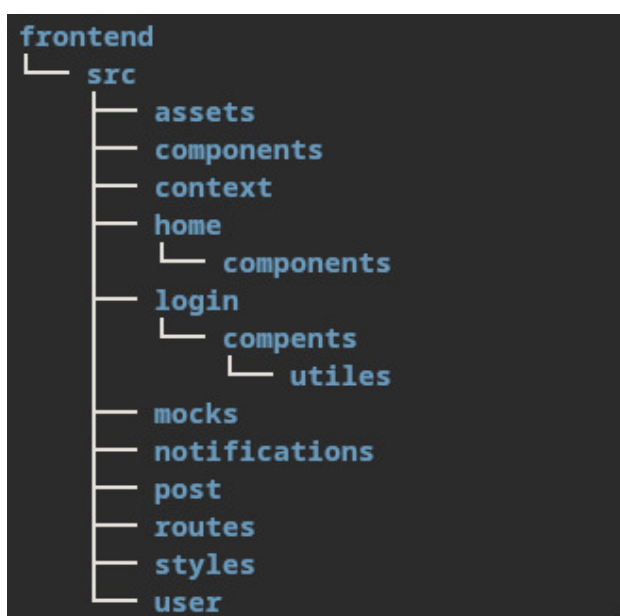


Figura 15: Estructura de carpetas

4.3.4.3 Arquitectura general del Frontend

El frontend de la aplicación se ha desarrollado utilizando React y TypeScript, siguiendo una arquitectura basada en componentes reutilizables y desacoplados. Esta estructura modular permite dividir la interfaz de usuario en bloques independientes, facilitando tanto el desarrollo como el mantenimiento y la escalabilidad del proyecto.

La navegación entre las diferentes vistas de la aplicación se gestiona mediante React Router, lo que permite implementar una Single Page Application (SPA) en la que el usuario puede desplazarse entre páginas sin recargar la web, mejorando así la experiencia de usuario y la eficiencia.

Para la gestión de estados globales, como la autenticación de usuario o el control del reproductor de música, se ha empleado la Context API de React. Esto permite compartir información y funcionalidades entre componentes distantes sin necesidad de prop drilling, manteniendo el código limpio y organizado.

El diseño visual y la experiencia de usuario se han implementado utilizando Material UI (MUI) y estilos personalizados en SCSS, garantizando una interfaz moderna, coherente y completamente responsiva. El uso de breakpoints y utilidades de MUI, junto con la personalización de estilos en SCSS, asegura que la aplicación se adapte correctamente a cualquier dispositivo, ya sea móvil o escritorio.

En resumen, la arquitectura del frontend se basa en la separación de responsabilidades, la reutilización de componentes y la gestión eficiente del estado global, lo que permite una evolución sencilla del proyecto y una experiencia de usuario óptima.

4.3.4.4 Consumo de APIs

El frontend de la aplicación se comunica con el backend a través de peticiones HTTP, utilizando principalmente la librería Axios para facilitar la gestión de las mismas. Todas las llamadas a la API están centralizadas en el archivo `api.ts`, donde se definen los métodos reutilizables para interactuar con los distintos endpoints del backend (autenticación, publicaciones, usuarios, notificaciones, etc.). Esta centralización permite mantener el código organizado, facilitar el mantenimiento y asegurar la coherencia en el tratamiento de las peticiones.

La gestión de las respuestas y los posibles errores se realiza de forma controlada: tras cada petición, se procesan los datos recibidos y se actualiza el estado de la interfaz de usuario en función del resultado. En caso de error, se muestran mensajes informativos o de advertencia al usuario, permitiendo una experiencia de uso fluida y segura. Además, se implementan mecanismos de control de sesión y autenticación, de modo que si se detecta un error relacionado con permisos o autenticación, la aplicación redirige automáticamente al usuario a la pantalla de login o muestra el mensaje correspondiente.



Esta arquitectura facilita la integración entre frontend y backend, mejora la mantenibilidad del código y permite una rápida adaptación ante posibles cambios en la API.

```
export const getHome = async () => {  
  const res = await fetchWithAuth('/api/home');  
  if (!res.ok) throw new Error('Error al obtener el feed');  
  return res.json();  
};
```

Figura 16: Ejemplo de función en archivo api.ts

4.3.4.5 Gestión de estados y contextos

Para la gestión eficiente de los estados globales en el frontend, la aplicación utiliza la Context API de React. Esta herramienta permite compartir información y funcionalidades entre componentes que no están directamente relacionados en la jerarquía, evitando el paso excesivo de props y facilitando la escalabilidad y el mantenimiento del código.

En la aplicación se han implementado principalmente dos contextos globales:

- **AuthContext:** Gestiona toda la información relacionada con la autenticación del usuario, como el estado de login, los datos del usuario autenticado y los métodos para iniciar o cerrar sesión. Gracias a este contexto, cualquier componente puede acceder de forma sencilla a la información del usuario y reaccionar ante cambios en la autenticación, permitiendo proteger rutas y mostrar contenido personalizado.
- **PlayerContextt:** Centraliza el estado y los controles del reproductor de música. Incluye la playlist actual, la canción seleccionada, el estado de reproducción, el volumen y la referencia global al elemento de audio. Esto permite que componentes como el reproductor, las tarjetas de canciones o las páginas de usuario puedan interactuar y sincronizarse fácilmente con el estado global del reproductor.

El uso de contextos aporta varias ventajas: simplifica la gestión de estados compartidos, mejora la organización del código, facilita la reutilización de lógica y componentes, y permite una comunicación eficiente entre distintas partes de la aplicación. De este modo, se consigue una arquitectura más limpia, escalable y fácil de mantener.

4.3.4.6 Componentes principales

El frontend de la aplicación está construido sobre una arquitectura de componentes reutilizables, cada uno de los cuales cumple una función específica dentro de la interfaz de usuario. Esta organi-



zación modular facilita la escalabilidad, el mantenimiento y la incorporación de nuevas funcionalidades en el futuro. A continuación, se describen los componentes más relevantes:

- **Login y Signup:** Estos componentes gestionan el acceso y registro de usuarios en la plataforma. El componente Login está estrechamente vinculado al sistema de autenticación, ya que se encarga de validar las credenciales del usuario mediante peticiones a la API y de gestionar el estado de autenticación a través del contexto global (AuthContext). Incluye formularios validados, gestión de errores, mensajes informativos y navegación automática según el estado de autenticación. Ambos componentes están diseñados para ser responsivos y ofrecer una experiencia de usuario sencilla y clara tanto en dispositivos móviles como en escritorio.
- **Home:** Es la página principal de la aplicación, donde se muestran las publicaciones musicales más recientes o destacadas. Utiliza el componente SongCard para renderizar cada publicación y permite al usuario interactuar con las canciones (reproducir, puntuar, comentar, guardar, etc.). Home también integra el MusicPlayer en su versión flotante para dispositivos móviles, facilitando la reproducción continua mientras se navega por la app.
- **SongCard:** Este componente es fundamental en la aplicación, ya que representa visualmente cada publicación musical. Muestra la portada, el título, el usuario que la ha publicado, la descripción y los botones de interacción (rate, comentar, compartir, guardar). Es reutilizable y se emplea en distintas secciones como Home, perfiles de usuario, resultados de búsqueda y la vista de detalle de una publicación.
- **MusicPlayer:** Gestiona la reproducción de audio en la aplicación. Permite al usuario reproducir, pausar, avanzar o retroceder entre canciones, ajustar el volumen y visualizar información sobre la canción actual. Está completamente integrado con el contexto global del reproductor, lo que garantiza la sincronización del estado de reproducción en toda la aplicación. Su diseño se adapta tanto a escritorio como a móvil, ofreciendo una experiencia consistente.
- **UserPage y OtherUserPage:** Son las páginas de perfil de usuario. UserPage muestra la información y publicaciones del usuario autenticado, permitiendo la gestión de su propio contenido, mientras que OtherUserPage permite visualizar el perfil y las publicaciones de otros usuarios. Ambas páginas incluyen secciones organizadas en pestañas (publicaciones, canciones guardadas, reposts y rates), facilitando la navegación y el acceso a la información relevante.
- **PostPage:** Muestra el detalle de una publicación concreta, permitiendo al usuario interactuar con la canción y acceder a información ampliada sobre la publicación y el usuario que la ha compartido.
- **SettingsPage:** Permite al usuario modificar su información personal, cambiar la contraseña, gestionar la privacidad y cerrar sesión. Incluye formularios validados y diálogos de confirmación para las acciones más sensibles.
- **NotificationList:** Gestiona y muestra las notificaciones recibidas por el usuario, como nuevos seguidores, comentarios, rates o interacciones en sus publicaciones. Facilita la interacción directa con las notificaciones y la navegación hacia el contenido relacionado.

- **NavigationWrapper:** Es el componente encargado de envolver la aplicación y proporcionar la estructura de navegación, incluyendo menús laterales, barra superior y el acceso a las distintas rutas. Gestiona la visibilidad y el comportamiento del menú en función del tamaño de pantalla y el estado de autenticación.
- **SearchBar:** Permite a los usuarios buscar canciones, publicaciones o perfiles dentro de la plataforma. Ofrece sugerencias en tiempo real y facilita el descubrimiento de nuevo contenido.

Todos estos componentes han sido diseñados siguiendo principios de modularidad y reutilización, lo que permite mantener una jerarquía clara y coherente. Además, se han aplicado estilos personalizados y variables globales en SCSS, así como utilidades de Material UI, para asegurar una experiencia visual atractiva y consistente en toda la aplicación. La relación y jerarquía entre estos componentes puede visualizarse mediante un diagrama de componentes, que ilustra cómo se integran y comunican dentro de la arquitectura general del frontend.

4.3.4.7 Estilos y responsividad

El diseño visual de la aplicación se ha gestionado combinando la librería Material UI (MUI) con hojas de estilo personalizadas en SCSS. Material UI proporciona una amplia variedad de componentes predefinidos y personalizables, lo que ha permitido mantener una interfaz moderna, coherente y profesional en todas las vistas de la aplicación. Además, se han utilizado los breakpoints y utilidades de MUI para adaptar automáticamente la disposición y el tamaño de los elementos a diferentes resoluciones de pantalla, garantizando así una experiencia de usuario óptima tanto en dispositivos móviles como en escritorio.

```
<Card
  ref={ref}
  onMouseEnter={() => setHover(true)}
  onMouseLeave={() => setHover(false)}
  onClick={handlePlay}
  sx={{
    width: largeSize
    ? { xs: '95vw', sm: '600px', md: '600px' }
    : mediumSize
    ? { xs: '95vw', sm: '450px', md: '450px' }
    : { xs: '90vw', sm: '350px', md: '400px' },
    height: largeSize
    ? { xs: '85vh', md: '720px' }
    : mediumSize
    ? { xs: '85vh', md: '580px' }
    : { xs: 'auto', sm: '380px', md: '540px' },
    display: 'flex',
    flexDirection: 'column',
    justifyContent: 'space-between',
    padding: '10px',
    boxShadow: 3,
    borderRadius: { xs: 2, md: 3 },
    position: 'relative',
    overflow: 'hidden',
    cursor: 'pointer',
    margin: { xs: 0, md: '16px' },
  }}
/>
```

Figura 17: Ejemplo de breakpoint en componente

Por otro lado, el uso de SCSS ha permitido definir variables globales (como colores principales, secundarios y de texto), así como estilos personalizados para componentes específicos, facilitando la reutilización y el mantenimiento del código de estilos. En el archivo global.scss se han centralizado los estilos generales, las fuentes y los colores corporativos, asegurando la coherencia visual en toda la aplicación.

La responsividad ha sido una prioridad durante el desarrollo, adaptando formularios, tarjetas, menú y el reproductor de música para que sean completamente funcionales y visualmente atractivos en cualquier dispositivo. Esto no solo mejora la accesibilidad, sino que también garantiza que todos los usuarios disfruten de una experiencia fluida y agradable, independientemente del tamaño de pantalla o el dispositivo utilizado.

4.3.4.8 Uso de mocks y documentación de la API

Durante el desarrollo del frontend, se ha empleado un enfoque profesional para facilitar la implementación y las pruebas, incluso en fases en las que el backend aún no estaba completamente disponible. Para ello, se ha utilizado un archivo de mocks (`mockSongs.ts`) que genera datos simulados de canciones y publicaciones. Este archivo ha permitido desarrollar y validar los componentes principales de la interfaz, como las listas de canciones, el reproductor y las tarjetas de publicaciones, sin depender de la disponibilidad real de los endpoints del backend.

Además, se ha elaborado un documento de referencia denominado API Map (`API_MAP.md`), que recoge de manera estructurada todos los endpoints disponibles en el backend, los métodos HTTP asociados, los parámetros requeridos y los formatos de las respuestas esperadas. Esta guía ha sido fundamental para coordinar el trabajo entre frontend y backend, asegurando que las peticiones y respuestas estuvieran correctamente alineadas y facilitando la integración final del sistema.

El uso de mocks y de una documentación clara de la API ha permitido avanzar en el desarrollo de forma paralela, detectar posibles incompatibilidades de forma temprana y garantizar una mayor calidad y robustez en la integración entre ambos lados de la aplicación.

4.3.4.9 Buenas prácticas

Durante el desarrollo del frontend se han seguido diversas buenas prácticas para garantizar la robustez, la calidad y la mantenibilidad del código. En primer lugar, se ha utilizado TypeScript como lenguaje principal, lo que aporta tipado estático y detección temprana de errores en tiempo de desarrollo. Esto facilita la identificación de posibles fallos antes de la ejecución, mejora la autocompletación y la documentación interna del código, y contribuye a la escalabilidad del proyecto a medida que crece en complejidad.

La aplicación se ha estructurado siguiendo principios de modularidad y reutilización. Cada funcionalidad principal se ha implementado como un componente independiente y reutilizable, lo que permite mantener una jerarquía clara y facilita la incorporación de nuevas características o la modificación de las existentes sin afectar al resto del sistema. Además, la separación de responsabilidades entre componentes, contextos y utilidades (como el archivo `api.ts` para las peticiones HTTP) contribuye a un código más limpio, organizado y fácil de mantener.

Por último, se han aplicado convenciones de nomenclatura coherentes, comentarios explicativos en las partes más complejas y una gestión centralizada de los estilos en archivos SCSS, como `global.scss`. Todo ello asegura que el proyecto sea fácilmente entendible y mantenible tanto por los desarrolladores actuales como por futuros colaboradores.

4.3.5. Pruebas funcionales del sistema

4.3.5.1 Backend

Para la validación del correcto funcionamiento del backend, se han realizado distintas pruebas funcionales sobre los principales endpoints de la API. Las pruebas se han llevado a cabo utilizando la herramienta de **Postman**, que permiten simular peticiones HTTP de forma controlada y verificar las respuestas del sistema y posteriormente utilizando el propio frontend de la aplicación.

A continuación se resumen algunos de los casos de uso probados, detallando el escenario de prueba, el resultado esperado y el resultado obtenido:

Caso de uso	Resultado esperado	Resultado obtenido
Registro de nuevo usuario	Usuario registrado correctamente y token JWT generado	Registro exitoso y token devuelto en respuesta
Inicio de sesión con credenciales correctas	Login exitoso y obtención de token JWT válido	Token JWT generado correctamente
Creación de publicación con álbum	Publicación creada y relaciones álbum-canciones establecidas	Publicación creada correctamente, datos persistidos
Recuperación de feed de inicio	Lista combinada de publicaciones y reposts de los usuarios seguidos	Feed devuelto correctamente ordenado por fecha
Subida de archivo multimedia válido	Archivo validado, almacenado y URL devuelta	Archivo aceptado y URL generada correctamente
Subida de archivo no permitido	Respuesta de error 400 Bad Request	Error 400 devuelto correctamente
Acceso a perfil de usuario	Información del usuario, estadísticas de publicaciones, seguidores y seguidos	Datos de perfil devueltos correctamente

Cuadro 3: Casos de prueba de validación de endpoints del backend(I)

Caso de prueba	Resultado esperado	Resultado obtenido
Intento de registro sin contraseña	Error de validación, respuesta 400	Error 400 devuelto correctamente
Intento de registro con email mal formado	Error de validación, respuesta 400	Error 400 devuelto correctamente
Acceso a recursos protegidos sin token	Error 401 Unauthorized	Error 401 devuelto correctamente
Acceso a recursos con token expirado	Error 401 Unauthorized	Error 401 devuelto correctamente
Acceso a recursos con token modificado	Error 401 Unauthorized	Error 401 devuelto correctamente
Creación de repost sobre publicación válida	Repost creado correctamente	Repost almacenado en base de datos
Creación de repost sobre publicación inexistente	Error de publicación no encontrada	Error devuelto correctamente
Valoración duplicada de una misma publicación	Error de duplicado o actualización de valoración	Gestión correcta de la lógica de negocio
Consulta de seguidores de un usuario existente	Lista de seguidores devuelta	Lista obtenida correctamente
Consulta de seguidos de un usuario existente	Lista de seguidos devuelta	Lista obtenida correctamente
Seguimiento de usuario válido	Relación de seguimiento creada	Follow registrado correctamente
Intento de seguir a un usuario inexistente	Error de validación o integridad	Error devuelto correctamente
Intento de seguirse a sí mismo	Error de validación de negocio	Error devuelto correctamente
Eliminación de un post existente	Publicación eliminada correctamente	Post eliminado en base de datos
Eliminación de un post inexistente	Error de publicación no encontrada	Error devuelto correctamente

Cuadro 4: Casos de prueba de validación de endpoints del backend(II)



4.3.5.2 Frontend

Preparación del entorno de pruebas

Para la realización de las pruebas funcionales, se preparó el entorno de desarrollo ejecutando el frontend localmente mediante el comando `npm start` en Visual Studio Code. El backend se desplegó tanto en local como en la máquina virtual Rocky Linux, permitiendo validar la integración real entre ambos entornos.

Durante las primeras fases, se emplearon datos simulados mediante el archivo `mockSongs.ts` para garantizar el correcto funcionamiento de los componentes principales incluso cuando el backend no estaba completamente disponible. Posteriormente, se configuraron las variables de entorno en el archivo `.env` para conectar el frontend con el backend real, asegurando la autenticación y el flujo de datos en tiempo real.

Adicionalmente, se utilizaron herramientas como Postman para comprobar manualmente los endpoints de la API y verificar la correcta respuesta del backend ante diferentes escenarios.

Resultados de las pruebas

Las pruebas funcionales se realizaron de forma manual, cubriendo los principales casos de uso de la aplicación. A continuación, se resumen los escenarios probados y los resultados obtenidos:

Caso de uso	Resultado esperado	Resultado obtenido
Inicio de sesión	El usuario accede con credenciales válidas	Correcto
Registro de nuevo usuario	Se crea una cuenta y se redirige a la home	Correcto
Cierre de sesión	El usuario sale de la sesión y vuelve a la pantalla de login	Correcto
Reproducción de canción	Se inicia la reproducción y se muestra el player	Correcto
Pausa y reanudación de canción	El usuario puede pausar y reanudar la reproducción	Correcto
Cambio de canción en playlist	El usuario puede avanzar o retroceder entre canciones	Correcto
Navegación entre páginas	No hay recarga, la navegación es fluida	Correcto
Guardar publicación	El icono cambia y la publicación se guarda	Correcto
Eliminar publicación	El post se elimina y desaparece de la lista	Correcto
Error de login	Se muestra mensaje de error si las credenciales fallan	Correcto
Visualización de perfil propio	Se muestra la información y publicaciones del usuario	Correcto
Visualización de perfil de otro usuario	Se muestra la información y publicaciones de otro usuario	Correcto
Edición de perfil	El usuario puede modificar su información personal y guardar los cambios	Correcto
Cambio de contraseña	El usuario puede cambiar su contraseña y recibe confirmación	Correcto

Cuadro 5: Resumen de pruebas funcionales realizadas en el frontend (I)

Caso de uso	Resultado esperado	Resultado obtenido
Cambio de correo electrónico	El usuario puede actualizar su correo y recibe confirmación	Correcto
Notificaciones	Se listan correctamente las notificaciones	Correcto
Creación de nueva publicación	El usuario puede subir una canción o álbum y aparece en la home	Correcto
Repost de publicación	El usuario puede hacer repost y la publicación aparece en su perfil	Correcto
Calificación de publicación	El usuario puede puntuar una publicación y la puntuación se actualiza	Correcto
Visualización de SongCard en móvil	El componente se adapta correctamente al tamaño de pantalla	Correcto
Visualización de SongCard en escritorio	El componente muestra mayor anchura y detalles visuales	Correcto
Hover sobre álbum en escritorio	Se muestra la lista de canciones del álbum correctamente	Correcto
Interacción con botones en SongCard	Todos los botones funcionan correctamente en cualquier tamaño de pantalla	Correcto
Búsqueda de canciones o usuarios	Se muestran resultados relevantes y navegación a los detalles	Correcto
Visualización de detalles de publicación	Se muestra información ampliada y comentarios	Correcto
Añadir comentario a publicación	El comentario se añade y aparece en la lista	Correcto
Gestión de estados globales (auth/player)	El usuario mantiene la sesión y el estado del reproductor entre páginas	Correcto
Visualización de álbum y reproducción de canciones individuales	Se puede seleccionar y reproducir cualquier canción del álbum	Correcto
Gestión de errores de red	Se muestra mensaje de error si falla la conexión con el backend	Correcto

Cuadro 6: Resumen de pruebas funcionales realizadas en el frontend (II)

4.3.6. Errores encontrados y resolución

4.3.6.1 Servidor y backend

Durante el desarrollo y despliegue del proyecto se han identificado diversos problemas técnicos que han requerido la aplicación de soluciones específicas. A continuación se resumen algunos de los principales errores detectados y las medidas adoptadas para su resolución:

- **Desbordamiento de pila de Hibernate por peticiones complejas**

Durante el desarrollo de algunas consultas avanzadas, especialmente aquellas que implicaban múltiples relaciones encadenadas en el modelo de entidades, Hibernate generaba árboles de carga de objetos excesivamente profundos, provocando el desbordamiento de la pila de ejecución (*stack overflow*) y la rotura de la transacción.

Solución: Se diseñaron vistas en SQL para precalcular y simplificar las consultas, descargando parte de la lógica en la base de datos y evitando así la necesidad de cargar estructuras excesivamente anidadas en Hibernate.

- **Errores CORS en peticiones desde el frontend**

Al realizar peticiones desde el frontend al backend, surgieron errores de política CORS (*Cross-Origin Resource Sharing*), bloqueando algunas comunicaciones entre el cliente y el servidor.

Solución: Se habilitó la configuración de CORS en el archivo `application.properties` de Quarkus, permitiendo los orígenes correspondientes del frontend.

- **Limitación de tamaño de petición en el backend**

Al implementar la funcionalidad de subida de archivos multimedia, se detectaron errores de limitación en el tamaño máximo permitido para los cuerpos de las peticiones HTTP.

Solución: Se ajustó la configuración de Quarkus incrementando el límite máximo de tamaño de petición (`quarkus.http.limits.max-body-size`) hasta los 50 MB, permitiendo así manejar los archivos de gran tamaño.

- **Limitación de tamaño de petición en el Ingress Controller**

De forma complementaria al ajuste en el backend, el *Ingress Controller* de Kubernetes también imponía límites en el tamaño máximo de las peticiones aceptadas, lo que generaba errores previos a la llegada de las peticiones al backend.

Solución: Se añadió la configuración `nginx.ingress.kubernetes.io/proxy-body-size: 50m` en el archivo YAML de configuración del Ingress, adaptando los límites al nuevo tamaño permitido.



■ Errores de eliminación recursiva en la base de datos

Durante operaciones de eliminación de registros relacionados (por ejemplo, eliminación de publicaciones vinculadas a álbumes o canciones), surgieron conflictos de integridad referencial por la existencia de claves foráneas.

Solución: Se añadió la cláusula `ON DELETE CASCADE` en las restricciones de las claves foráneas de las tablas afectadas, permitiendo así la eliminación recursiva automática de los registros dependientes.

4.3.6.2 Frontend

Error encontrado	Descripción	Solución aplicada
Reproductor no sincronizaba la playlist	Al reproducir un álbum, el reproductor no actualizaba correctamente la lista de canciones ni el índice de reproducción, impidiendo avanzar entre canciones del álbum.	Se revisó la lógica del contexto global del reproductor, asegurando que al seleccionar un álbum se cargara correctamente la playlist y el índice inicial. Se añadieron logs y pruebas manuales para validar el comportamiento.
Problemas con referencias de audio	El elemento <audio> no respondía correctamente a los controles de reproducción o daba errores al acceder a la referencia desde distintos componentes.	Se centralizó la gestión de la referencia de audio en el contexto global y se revisó el uso de <code>useRef</code> y <code>forwardRef</code> , garantizando una referencia única y accesible.
Errores de responsividad en SongCard	SongCard no se adaptaba correctamente a todos los tamaños de pantalla, provocando desbordamientos o solapamientos de elementos.	Se ajustaron los estilos utilizando los breakpoints de MUI en la propiedad <code>sx</code> , y se realizaron pruebas manuales en distintos dispositivos y resoluciones.
Eventos no deseados al interactuar con botones	Al hacer clic en botones internos (guardar, repostear, borrar) dentro de SongCard, se activaba también el evento de reproducción.	Se añadió <code>e.stopPropagation()</code> en los manejadores de eventos de los botones internos para evitar la propagación al contenedor principal.

Cuadro 7: Errores significativos detectados durante el desarrollo y su resolución (I)

Error encontrado	Descripción	Solución aplicada
Problemas de integración frontend-backend	Las respuestas de la API no coincidían con la estructura esperada por el frontend, generando errores de renderizado o fallos en la lógica.	Se actualizó el archivo <code>API_MAP.md</code> , se adaptaron los mocks y el manejo de respuestas en el frontend para asegurar la compatibilidad.
Gestión de estados inconsistentes	Tras eliminar o guardar publicaciones, la interfaz no se actualizaba correctamente hasta recargar la página.	Se mejoró la gestión de estados locales y globales, actualizando las listas y componentes afectados tras cada acción mediante hooks y funciones de actualización de estado.
Errores de autenticación y expiración de sesión	Si el token expiraba o el usuario perdía la sesión, la aplicación no redirigía correctamente al login ni mostraba mensajes claros.	Se implementó una gestión centralizada de errores de autenticación en el contexto de usuario, forzando el logout y la redirección al login en caso de error 401 o expiración de sesión.

Cuadro 8: Errores significativos detectados durante el desarrollo y su resolución (II)

5. Conclusiones

El desarrollo de Vibra ha permitido construir un sistema completo, funcional y profesional, en el que se han abordado de forma integral tanto los aspectos de backend, frontend como los de infraestructura y despliegue.

Desde el lado del backend, el uso de **Quarkus**, **Hibernate Reactive Panache** y la programación reactiva con **Mutiny** ha permitido construir un servicio altamente eficiente, preparado para entornos de alta concurrencia y plenamente optimizado para ejecución en contenedores. La adopción de un modelo asíncrono ha supuesto un reto técnico importante, especialmente a la hora de gestionar operaciones complejas de acceso a datos y sincronización de operaciones dependientes. La utilización de vistas SQL para descargar parte de la lógica hacia la base de datos **PostgreSQL** ha sido clave para resolver problemas de rendimiento y mantener la eficiencia del sistema. Además, se han implementado mecanismos robustos de seguridad mediante **JWT**, validaciones centralizadas y una completa gestión de archivos multimedia, garantizando la integridad y la seguridad de la aplicación.

En cuanto a la infraestructura de despliegue, se ha construido un entorno completamente funcional sobre una máquina **Rocky Linux minimal**, utilizando **Kubernetes** a través de **Kind**, y gestionando los contenedores con **Podman**. Este entorno ha permitido replicar de forma fiel las condiciones de un sistema de producción, facilitando tanto las pruebas como la posterior escalabilidad del proyecto. La gestión de los recursos mediante manifiestos **YAML**, la configuración de los **secrets** para el sistema de autenticación, y el control fino de las políticas de red y volúmenes de almacenamiento han proporcionado un despliegue robusto, controlado y fácilmente mantenible.

En el frontend, el proyecto ha servido como un entorno real de aplicación de conocimientos avanzados en **React**, **TypeScript** y **Material UI**. La arquitectura modular basada en componentes reutilizables y la gestión eficiente del estado global mediante **Context API** han facilitado tanto el desarrollo como el mantenimiento. Gracias a la documentación temprana de la API y al uso de **mocks** durante el desarrollo inicial, fue posible realizar un trabajo paralelo con el backend minimizando los tiempos de integración. La aplicación resultante ofrece una interfaz moderna, intuitiva y completamente adaptada a diferentes dispositivos, permitiendo al usuario disfrutar de una experiencia fluida y coherente.

A nivel global, el proyecto ha cumplido satisfactoriamente los objetivos establecidos al inicio, abarcando todas las capas de desarrollo e integración de un sistema moderno basado en microservicios y desplegado en un entorno controlado de producción. Además, el trabajo realizado ha permitido consolidar conocimientos avanzados en programación reactiva, gestión de infraestructura *cloud-native*, desarrollo frontend moderno y metodologías de integración continua. **Vibra** queda así como una plataforma sólida, bien estructurada y preparada para su futura evolución.



6. Bibliografía

Referencias

- [1] React, “React – A JavaScript library for building user interfaces.” [En línea]. Disponible en: <https://react.dev/>. [Accedido: 11-jun-2025].
- [2] Material UI, “Material UI – React components for faster and easier web development.” [En línea]. Disponible en: <https://mui.com/>. [Accedido: 11-jun-2025].
- [3] TypeScript, “TypeScript – JavaScript with syntax for types.” [En línea]. Disponible en: <https://www.typescriptlang.org/>. [Accedido: 11-jun-2025].
- [4] Axios, “Axios – Promise based HTTP client for the browser and node.js.” [En línea]. Disponible en: <https://axios-http.com/>. [Accedido: 11-jun-2025].
- [5] Node.js, “Node.js – JavaScript runtime built on Chrome’s V8 JavaScript engine.” [En línea]. Disponible en: <https://nodejs.org/>. [Accedido: 11-jun-2025].
- [6] Stack Overflow, “Stack Overflow – Where Developers Learn, Share, Build Careers.” [En línea]. Disponible en: <https://stackoverflow.com/>. [Accedido: 11-jun-2025].
- [7] Quarkus, “Quarkus - Creating Your First Application” [En línea]. Disponible en: <https://quarkus.io/guides/getting-started>.
- [8] Quarkus, “Quarkus - Getting Started With Reactive” [En línea]. Disponible en: <https://quarkus.io/guides/getting-started-reactive>.
- [9] Quarkus, “Quarkus - OpenID Connect client and token propagation ” [En línea]. Disponible en: <https://quarkus.io/guides/security-openid-connect-client>.
- [10] Quarkus, “Quarkus - Getting started with security” [En línea]. Disponible en: <https://quarkus.io/guides/security-getting-started-tutorial>.
- [11] Red Hat, “Red Hat - Reactive in Red Hat Build of Quarkus” [En físico]. No disponible.
- [12] Red Hat, “Red Hat - Securing Microservices and Role Based Access Control by using JSON Web Tokens” [En físico]. No disponible.
- [13] Red Hat, “Red Hat - Developing Cloud-native Microservices with Quarkus” [En físico]. No disponible.
- [14] Kind, “Kind - Quick Start” [En línea]. Disponible en: <https://kind.sigs.k8s.io/docs/user/quick-start/>.
- [15] Kind, “Kind - Configuration” [En línea]. Disponible en: <https://kind.sigs.k8s.io/docs/user/configuration/>.
- [16] Kind, “Kind - Ingress” [En línea]. Disponible en: <https://kind.sigs.k8s.io/docs/user/ingress/>.